Date Published: April 01, 2026

Visit the Zebra Product Support page for the most up-to-date version of this documentation.

M3dgeo functions

Synopsis

The functions prefixed with M3dgeo are responsible for the generation of 3D geometries such as 3D axis-aligned box, 3D sphere, and 3D plane. You can also generate transformation matrices (used to describe 3D transformations such as translation and rotation).

Functions

M3dgeoAlloc
M3dgeoBox
M3dgeoConstruct
M3dgeoCopy
M3dgeoCylinder
M3dgeoDraw3d
M3dgeoEvalCurve
M3dgeoEvalSurface
M3dgeoFree
M3dgeoInquire
M3dgeoLine
M3dgeoMatrixGet
M3dgeoMatrixGetTransform
M3dgeoMatrixPut
M3dgeoMatrixSetTransform
M3dgeoMatrixSetWithAxes
M3dgeoPlane
M3dgeoPoint
M3dgeoRestore
M3dgeoSave
M3dgeoSphere
M3dgeoStream

M3dgeoAlloc

Synopsis

Allocate a 3D geometry object or transformation matrix object.

Syntax

```
AIL_ID M3dgeoAlloc(
    AIL_ID SysId,                      //in
    AIL_INT64 ObjectType,              //in
    AIL_INT64 ControlFlag,             //in
    AIL_ID *GeometryOrMatrix3dgeoIdPtr //out
)
```

Description

This function allocates a 3D geometry object or transformation matrix object on the specified system.

Note that 3D geometry objects or transformation matrix objects allocated with **M3dgeoAlloc**() can be saved using either M3dgeoStream() or M3dgeoSave() and restored using M3dgeoRestore().

When the 3D geometry object or transformation matrix object is no longer required, release it using M3dgeoFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the 3D geometry object or transformation matrix object's lifetime and you must not manually free it.

Parameters

SysId

Specifies the identifier of the system on which to allocate the 3D geometry object or transformation matrix object.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ObjectType

Specifies to allocate a 3D geometry object or transformation matrix object. This parameter must be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the object type

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_GEOMETRY	Allocates a 3D geometry object on the specified system.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRANSFORMATION_MATRIX	Allocates a transformation matrix object on the specified system.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

GeometryOrMatrix3dgeoIdPtr

Specifies the address of the variable in which to write the 3D geometry object or transformation matrix object identifier or specifies the data type that the function should use to return the 3D geometry object or transformation matrix object identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated 3D geometry object or transformation matrix object ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated 3D geometry object or transformation matrix object ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_3DGEO_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the 3D geometry object or transformation matrix object (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the 3D geometry object identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated 3D geometry object identifier. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Address in which to write the transformation matrix object identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated transformation matrix object identifier. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b	

Return value

The returned value is the 3D geometry object or transformation matrix object identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_3DGEO_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3d11.lib.
DLL	Requires ail11.dll; ail3d11.dll.

M3dgeoBox

Synopsis

Define a 3D box geometry object.

Syntax

```
void M3dgeoBox(
    AIL_ID Geometry3dgeoId,   //out
    AIL_INT64 CreationMode,   //in
    AIL_DOUBLE XPos1,         //in
    AIL_DOUBLE YPos1,         //in
    AIL_DOUBLE ZPos1,         //in
    AIL_DOUBLE XPos2OrLength, //in
    AIL_DOUBLE YPos2OrLength, //in
    AIL_DOUBLE ZPos2OrLength, //in
    AIL_INT64 ControlFlag     //in
)
```

Description

This function defines an axis-aligned, 3D box geometry object. You can get or set the 3D box orientation using M3dgeoCopy(). You can translate, rotate, scale, or transform the resulting box, using the 3D image processing module.

You can use the resulting box to, for example, crop a point cloud or perform an arithmetic operation on a depth map using the 3D image processing module, or calculate its distance from each point in a point cloud using the 3D metrology module.

If you want to define a box geometry object from results obtained in a different module, you can use the copy function of that module.

All coordinates are expressed in world units in the working coordinate system.

> Note that if Geometry3dgeoId specifies a previously-defined box, and CreationMode is set to M_CENTER_AND_DIMENSION, you can leave some of the box's attributes unchanged, even if that attribute was set using a different creation mode or was modified using the 3D image processing module. Unless you specify M_ORIENTATION_UNCHANGED, its orientation will be reset to the identity matrix.

Parameters

Geometry3dgeoId

Specifies the identifier of the 3D geometry object, previously allocated on the required system using M3dgeoAlloc() with M_GEOMETRY.

CreationMode

Specifies how the box is defined.

See the Parameter associations section for possible values that can be specified.

XPos1

Specifies the X-coordinate of the first point used to define the box.

See the Parameter associations section for possible values that can be specified.

YPos1

Specifies the Y-coordinate of the first point used to define the box.

See the Parameter associations section for possible values that can be specified.

ZPos1

Specifies the Z-coordinate of the first point used to define the box.

See the Parameter associations section for possible values that can be specified.

XPos2OrLength

Specifies the X-coordinate of the second point used to define the box, or the length of the box along the X-axis.

See the Parameter associations section for possible values that can be specified.

YPos2OrLength

Specifies the Y-coordinate of the second point used to define the box, or the length of the box along the Y-axis.

See the Parameter associations section for possible values that can be specified.

ZPos2OrLength

Specifies the Z-coordinate of the second point used to define the box, or the length of the box along the Z-axis.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Parameter associations

The table below lists possible values for the **CreationMode**, **XPos1**, **YPos1**, **ZPos1**, **XPos2OrLength**, **YPos2OrLength**, and **ZPos2OrLength** parameters.

- For specifying the box

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the box INQ																		
CreationMode		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	XPos1																
	YPos1																
	ZPos1																

	XPos2OrLength																
	YPos2OrLength																
	ZPos2OrLength																
M_BOTH_CORNERS		Defines the box using any two opposite corners.															
	XPos1	Specifies the X-coordinate of the first corner of the box.	c		e	f	g	a		i		j	k	l	m	n	b
	YPos1	Specifies the Y-coordinate of the first corner of the box.	c		e	f	g	a		i		j	k	l	m	n	b
	ZPos1	Specifies the Z-coordinate of the first corner of the box.	c		e	f	g	a		i		j	k	l	m	n	b
	XPos2OrLength	Specifies the X-coordinate of the second corner of the box, opposite to the first corner.	c		e	f	g	a		i		j	k	l	m	n	b
	YPos2OrLength	Specifies the Y-coordinate of the second corner of the box, opposite to the first corner.	c		e	f	g	a		i		j	k	l	m	n	b
	ZPos2OrLength	Specifies the Z-coordinate of the second corner of the box, opposite to the first corner.	c		e	f	g	a		i		j	k	l	m	n	b

M_CENTER_AND_DIMENSION +	Defines the box by its center point and its length along each axis. You can determine the coordinates of the corners of the box by taking a coordinate from the center point and adding or subtracting half the box's length, along the coordinate's axis. For example, the Y-coordinates of the corners of the box are YPos1 ± (YPos2OrLength /2). You can also use M3dgeoInquire() with M_CORNER_.... Note that the box's size along each axis, M_SIZE_X, M_SIZE_Y, and M_SIZE_Z, will be the absolute values of the corresponding lengths specified by XPos2OrLength, YPos2OrLength, and ZPos2OrLength. Note that if Geometry3dgeoId specifies a previously-defined box, you can leave some of its attributes unchanged, even if that attribute was set using a different creation mode or was modified using the 3D image processing module. To do so, set the corresponding parameter to M_UNCHANGED.	c		e	f	g	a		i		j	k	l	m	n	b
XPos1	Specifies the X-coordinate of the center of the box.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the box center's X-coordinate.	c		e	f	g	a		i		j	k	l	m	n	b
YPos1	Specifies the Y-coordinate of the center of the box.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the box center's Y-coordinate.	c		e	f	g	a		i		j	k	l	m	n	b
ZPos1	Specifies the Z-coordinate of the center of the box.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b

		Value	Specifies the box center's Z-coordinate.	c		e	f	g	a		i		j	k	l	m	n	b
	XPos2OrLength		Specifies the length of the box along the X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
		M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the box's length along the X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
	YPos2OrLength		Specifies the length of the box along the Y-axis.	c		e	f	g	a		i		j	k	l	m	n	b
		M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the box's length along the Y-axis.	c		e	f	g	a		i		j	k	l	m	n	b
	ZPos2OrLength		Specifies the length of the box along the Z-axis.	c		e	f	g	a		i		j	k	l	m	n	b
		M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the box's length along the Z-axis.	c		e	f	g	a		i		j	k	l	m	n	b

M_CORNER_AND_DIMENSION		Defines the box by one of its corners and its length (or negative length) along each axis. A positive value for length extends the box in the direction of the positive axis, starting from the specified corner point. A negative value extends the box in the direction of the negative axis, starting at the specified corner point. For example, if the specified corner is (1, 1, 1) and the value of the X, Y, and Z lengths are all positive 3 (a cube), the coordinates of the corner opposite the specified corner would be (4, 4, 4). If the X length was -3, while the other two lengths remained positive 3 (still a cube), the coordinates of the corner opposite the specified corner would be (-2, 4, 4). Note that the box's size along each axis, M_SIZE_X, M_SIZE_Y, and M_SIZE_Z, will be the absolute values of the corresponding lengths specified by XPos2OrLength, YPos2OrLength, and ZPos2OrLength.	c		e	f	g	a		i		j	k	l	m	n	b
	XPos1	Specifies the X-coordinate of one corner of the box.	c		e	f	g	a		i		j	k	l	m	n	b
	YPos1	Specifies the Y-coordinate of one corner of the box.	c		e	f	g	a		i		j	k	l	m	n	b
	ZPos1	Specifies the Z-coordinate of one corner of the box.	c		e	f	g	a		i		j	k	l	m	n	b
	XPos2OrLength	Specifies the length of the box along the X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
	YPos2OrLength	Specifies the length of the box along the Y-axis.	c		e	f	g	a		i		j	k	l	m	n	b
	ZPos2OrLength	Specifies the length of the box along the Z-axis.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_CENTER_AND_DIMENSION.

You can add the following value to the above-mentioned value to specify to leave the orientation of the previously-defined box unchanged.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying to leave the orientation unchanged																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ORIENTATION_UNCHANGED	Specifies to leave the orientation of the previously-defined box unchanged.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3d11.lib.
DLL	Requires ail11.dll; ail3d11.dll.

M3dgeoConstruct

Synopsis

Construct a 3D geometry from other geometries.

Syntax

```
void M3dgeoConstruct(
    AIL_ID Src1Geometry3dgeoId, //in
    AIL_ID Src2Geometry3dgeoId, //in
    AIL_ID DstGeometry3dgeoId,  //out
    AIL_INT GeometryType,       //in
    AIL_INT64 Operation,        //in
    AIL_DOUBLE Param,           //in
    AIL_INT64 ControlFlag       //in
)
```

Description

This function constructs a new 3D geometry using the geometry from one or two source 3D geometries. You can construct a box, a cylinder, a line, a plane, a point, or a sphere.

Parameters

Src1Geometry3dgeoId

Specifies the first source 3D geometry object.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the first source 3D geometry																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_XY_PLANE	Specifies the XY (Z=0) plane.	c		e	f	g	a		i		j	k	l	m	n	b
3D geometry object identifier	Specifies the identifier of a 3D geometry object. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and must have been successfully defined.	c		e	f	g	a		i		j	k	l	m	n	b

Src2Geometry3dgeoId

Specifies the second source 3D geometry object. If a second source is not required, set this parameter to **M_NULL**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the second source 3D geometry																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_XY_PLANE	Specifies the XY (Z=0) plane.	c		e	f	g	a		i		j	k	l	m	n	b
3D geometry object identifier	Specifies the identifier of a 3D geometry object. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and must have been successfully defined.	c		e	f	g	a		i		j	k	l	m	n	b

DstGeometry3dgeoId

Specifies the identifier of the destination 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY.

GeometryType

Specifies the type of 3D geometry to construct.

See the Parameter associations section for possible values that can be specified.

Operation

Specifies how to construct the 3D geometry.

See the Parameter associations section for possible values that can be specified.

Param

Specifies a scalar value for certain operations, where required. Set this parameter to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Parameter associations

The table below lists possible values for the **GeometryType**, **Operation**, and **Param** parameters.

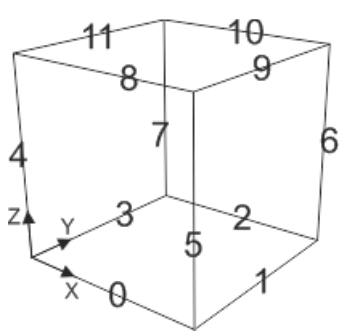
- For specifying the type of geometry to construct

Set unused parameters to **M_DEFAULT**.

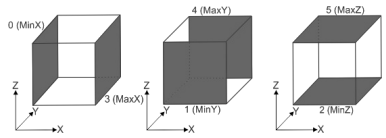
Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

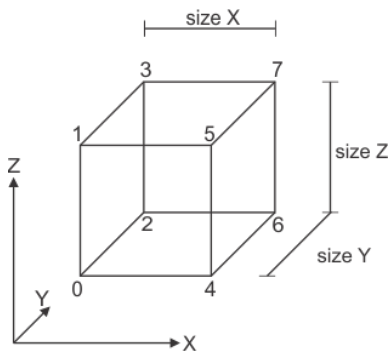
For specifying the type of geometry to construct																	
GeometryType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	Operation																
	Param																
M_BOX		Specifies to construct a box.															

Operation	Specifies how the box is defined.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOTH_CORNERS	Constructs the box using two opposite corners. Src1Geometry3dgeoId and Src2Geometry3dgeoId must be defined as points.	c		e	f	g	a		i		j	k	l	m	n	b
M_CYLINDER	Specifies to construct a cylinder.	c		e	f	g	a		i		j	k	l	m	n	b
Operation	Specifies how the cylinder is defined.	c		e	f	g	a		i		j	k	l	m	n	b
M_AXIS	Constructs a cylinder using a central axis and a radius. Src1Geometry3dgeoId determines the central axis and must be defined as a line.	c		e	f	g	a		i		j	k	l	m	n	b
M_FLIP	Constructs a cylinder by flipping the specified cylinder in on itself by swapping its start and end points. Src1Geometry3dgeoId must be defined as a cylinder.	c		e	f	g	a		i		j	k	l	m	n	b
M_TWO_POINTS	Constructs a cylinder using two points and a radius. Src1Geometry3dgeoId and Src2Geometry3dgeoId must be defined as points.	c		e	f	g	a		i		j	k	l	m	n	b
Param	Specifies the cylinder's radius, for an M_AXIS or an M_TWO_POINTS operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies that this parameter is not used.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Sets the radius of the cylinder.	c		e	f	g	a		i		j	k	l	m	n	b
M_LINE	Specifies to construct a line.	c		e	f	g	a		i		j	k	l	m	n	b
Operation	Specifies how the line is defined.	c		e	f	g	a		i		j	k	l	m	n	b
M_AXIS	Constructs a line on the specified cylinder's central axis; the line will have the same length as the cylinder. Src1Geometry3dgeoId must be defined as a cylinder.	c		e	f	g	a		i		j	k	l	m	n	b

		M_EDGE	Constructs a line on the specified edge of the source box geometry. The line will have the same length as the edge. Src1Geometry3dgeoId must be defined as a box.	c		e	f	g	a		i		j	k	l	m	n	b
		M_FLIP	Constructs a line by flipping the specified line in on itself by swapping its start and end points. Src1Geometry3dgeoId must be defined as a line.	c		e	f	g	a		i		j	k	l	m	n	b
		M_NORMAL	Constructs a unit line parallel to the plane's normal. Src1Geometry3dgeoId must be defined as a plane, and Src2Geometry3dgeoId must be defined as a point or **M_NULL**. If Src2Geometry3dgeoId is **M_NULL**, the line passes through the closest point to the origin; otherwise, it passes through Src2Geometry3dgeoId.	c		e	f	g	a		i		j	k	l	m	n	b
		M_TWO_POINTS	Constructs a finite line from two points. Src1Geometry3dgeoId and Src2Geometry3dgeoId must be defined as points.	c		e	f	g	a		i		j	k	l	m	n	b
	Param		Specifies the box's edge, for an M_EDGE operation.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Specifies that this parameter is not used.	c		e	f	g	a		i		j	k	l	m	n	b
		0 <= *Value* <= 11	Sets the edge along which to construct the line. The edges are assigned the following indices upon initially defining the box, based on the edge's position with respect to the axes of the working coordinate system. The index of an edge does not change based on the orientation of the box.  	c		e	f	g	a		i		j	k	l	m	n	b

M_PLANE	Specifies to construct a plane.	c		e	f	g	a		i		j	k	l	m	n	b
Operation	Specifies how the plane is defined.	c		e	f	g	a		i		j	k	l	m	n	b
M_FACE	Constructs a plane passing through the face of a box or cylinder. The plane's normal always points outside the geometry. Src1Geometry3dgeoId must be defined as a box or cylinder. Use Param to specify the box's face or cylinder's base. For infinite cylinders, set Param to 0, which constructs the plane at the cylinder's start point, perpendicular to its central axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_FLIP	Constructs a plane by flipping the specified plane in on itself by inverting its normal vector. Src1Geometry3dgeoId must be defined as a plane.	c		e	f	g	a		i		j	k	l	m	n	b
M_LINE_AND_POINT	Constructs a plane passing through the specified line and point. Src1Geometry3dgeoId must be defined as a line, and Src2Geometry3dgeoId must be defined as a point. You can use M_LINE_AND_POINT to construct a plane from 3 points if you first construct a line using two of the points (M_TWO_POINTS).	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMAL	Constructs a plane with a normal vector that coincides with the specified line. Src1Geometry3dgeoId must be defined as a line, and Src2Geometry3dgeoId must be defined as a point or **M_NULL**. The plane passes through the specified point (Src2Geometry3dgeoId); otherwise, the plane passes through the line's start point.	c		e	f	g	a		i		j	k	l	m	n	b
Param	Specifies the plane's position, for an M_FACE operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies that this parameter is not used.	c		e	f	g	a		i		j	k	l	m	n	b
0	Specifies that the plane passes through the start point of the infinite cylinder.	c		e	f	g	a		i		j	k	l	m	n	b

		0 <= *Value* <= 1	Specifies either the first (0) or the second (1) circular base of the cylinder.	c		e	f	g	a		i		j	k	l	m	n	b
		0 <= *Value* <= 5	Specifies the face of the box. The faces are assigned the following indices upon initially defining the box, based on the face's position with respect to the axes of the working coordinate system. The index of a face does not change based on the orientation of the box.  To inquire the box's minimum and maximum coordinates, use M3dgeoInquire() with M_UNROTATED_....	c		e	f	g	a		i		j	k	l	m	n	b
M_POINT			Specifies to construct a point.	c		e	f	g	a		i		j	k	l	m	n	b
	Operation		Specifies how the point is defined.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CENTER	Constructs a point at the center of the specified 3D geometry. Src1Geometry3dgeoId must be defined as a box, finite cylinder, finite line, point, or sphere.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CLOSEST_TO_ORIGIN	Constructs a point on the plane such that the point is closest to the origin. Src1Geometry3dgeoId must be defined as a plane.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CORNER	Constructs a point at the specified corner of the box. Src1Geometry3dgeoId must be defined as a box.	c		e	f	g	a		i		j	k	l	m	n	b
		M_END_POINT	Constructs a point at the end point of a finite line or a finite cylinder. Src1Geometry3dgeoId must be defined as a finite line or finite cylinder.	c		e	f	g	a		i		j	k	l	m	n	b
		M_START_POINT	Constructs a point at the start point of a line or cylinder. Src1Geometry3dgeoId must be defined as a line or cylinder.	c		e	f	g	a		i		j	k	l	m	n	b

Param	Specifies a corner on the box, for an M_CORNER operation.	c	e	f	g	a	i	j	k	l	m	n	b
M_DEFAULT	Specifies that this parameter is not used.	c	e	f	g	a	i	j	k	l	m	n	b
0 <= *Value* <= 7	Specifies the box corner. 	c	e	f	g	a	i	j	k	l	m	n	b
M_SPHERE	Specifies to construct a sphere.	c	e	f	g	a	i	j	k	l	m	n	b
Operation	Specifies how the sphere is defined.	c	e	f	g	a	i	j	k	l	m	n	b
M_CENTER_AND_RADIUS	Constructs a sphere using a center point and a radius. Src1Geometry3dgeoId must be defined as a point.	c	e	f	g	a	i	j	k	l	m	n	b
M_DIAMETER	Constructs a sphere using two antipodal points. Src1Geometry3dgeoId and Src2Geometry3dgeoId must be defined as points.	c	e	f	g	a	i	j	k	l	m	n	b
Param	Specifies the sphere's radius, for an M_CENTER_AND_RADIUS operation.	c	e	f	g	a	i	j	k	l	m	n	b
M_DEFAULT	Specifies that this parameter is not used.	c	e	f	g	a	i	j	k	l	m	n	b
Value >= 0.0	Sets the radius of the sphere.	c	e	f	g	a	i	j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3d11.lib.
DLL	Requires ail11.dll; ail3d11.dll.

M3dgeoCopy

Synopsis

Copy attributes or settings to or from a 3D geometry object or transformation matrix object.

Syntax

```
void M3dgeoCopy(
    AIL_ID SrcAilObjectId, //in
    AIL_ID DstAilObjectId, //out
    AIL_INT64 CopyType,    //in
    AIL_INT64 ControlFlag  //in
)
```

Description

This function copies attributes or settings to or from a 3D geometry object or transformation matrix object. You can copy the entire object or some attributes or settings of the object (for example, box orientation), depending on the source and destination objects and the specified copy type.

Parameters

SrcAilObjectId

Specifies the identifier of the source object. Note that if the source object is not a predefined or user-defined 3D geometry object or transformation matrix object, the destination object must be.

See the Parameter associations section for possible values that can be specified.

DstAilObjectId

Specifies the identifier of the destination object. Note that if the destination object is not a predefined or user-defined 3D geometry object or transformation matrix object, the source object must be.

See the Parameter associations section for possible values that can be returned.

CopyType

Specifies the type of copy operation.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Parameter associations

The table below lists possible values for the **SrcAilObjectId** and **CopyType** parameters and possible values returned to the **DstAilObjectId** parameter.

- For specifying the copy type

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the copy type																	
CopyType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
SrcAilObjectId																
DstAilObjectId - Possible values returned																
M_BOX_ORIENTATION	Specifies to set the orientation of a 3D box geometry object, or copy the orientation of a 3D box geometry object. Note that changing the box orientation will not affect its center or its dimensions (that is, the box rotates around its center).															
M_IDENTITY_MATRIX	Specifies to set the 3D box geometry object's box orientation to the identity matrix. This can be used to reset the box's orientation.	c		e	f	g	a		i		j	k	l	m	n	b
3D geometry object ID	Specifies the identifier of the 3D box geometry object whose orientation to set. The 3D box geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box.	c		e	f	g	a		i		j	k	l	m	n	b
3D box geometry object ID from which to copy	Specifies the identifier of a 3D box geometry object from which to copy the box orientation. The 3D box geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box.	c		e	f	g	a		i		j	k	l	m	n	b
3D box geometry object ID in which to copy	Specifies the identifier of the 3D box geometry object whose orientation to set. The 3D box geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box.	c		e	f	g	a		i		j	k	l	m	n	b

Transformation matrix object ID in which to copy	Specifies the identifier of the transformation matrix object in which to copy the box orientation. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
Transformation matrix object ID to use to define	Specifies the identifier of the transformation matrix object used to set the 3D box geometry object's box orientation. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
3D geometry object ID	Specifies the identifier of the 3D box geometry object whose orientation to set. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY.	c		e	f	g	a		i		j	k	l	m	n	b
M_GEOMETRY	Specifies to copy a 3D geometry object into another 3D geometry object.	c		e	f	g	a		i		j	k	l	m	n	b
M_XY_PLANE	Specifies to copy the XY (Z=0) plane into a 3D geometry object.	c		e	f	g	a		i		j	k	l	m	n	b
3D geometry object ID	Specifies the identifier of the 3D geometry object whose geometry to set to the XY (Z=0) plane. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY.	c		e	f	g	a		i		j	k	l	m	n	b
3D geometry object ID from which to copy	Specifies the identifier of a 3D geometry object from which to copy the geometry. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and must have been successfully defined.	c		e	f	g	a		i		j	k	l	m	n	b

		3D geometry object ID	Specifies the identifier of the 3D geometry object whose geometry to set. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_AND_SCALE			Specifies to set only the rotation and scale values of a transformation matrix object using an Aurora Imaging Library array buffer, or to copy only the rotation and scale values of a transformation matrix object.	c		e	f	g	a		i		j	k	l	m	n	b
	M_IDENTITY_MATRIX		Specifies to copy the rotation and scale values of the identity matrix (equivalent to no rotation and uniform scale of 1).	c		e	f	g	a		i		j	k	l	m	n	b
		Array buffer identifier in which to copy	Specifies the identifier of the 4x4 Aurora Imaging Library array buffer in which to copy the rotation and scale values of the identity matrix. The Aurora Imaging Library array buffer must have been previously allocated using MbufAlloc2d() with M_ARRAY and 32 + M_FLOAT.	c		e	f	g	a		i		j	k	l	m	n	b
		Transformation matrix object ID in which to copy	Specifies the identifier of the destination transformation matrix object in which to copy the rotation and scale values of the identity matrix. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b

	Array buffer identifier to use to define	Specifies the identifier of a 4x4 Aurora Imaging Library array buffer used to set the rotation and scale values of the transformation matrix object. The Aurora Imaging Library array buffer must have been previously allocated using MbufAlloc2d() with M_ARRAY and 32 + M_FLOAT, and must have been initialized.	c		e	f	g	a		i		j	k	l	m	n	b
	Transformation matrix object ID	Specifies the identifier of the transformation matrix object. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
	Transformation matrix object ID from which to copy	Specifies the identifier of a transformation matrix object from which to copy the rotation and scale values. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
	Array buffer identifier in which to copy	Specifies the identifier of the 4x4 Aurora Imaging Library array buffer in which to copy the rotation and scale values of the transformation matrix object. The Aurora Imaging Library array buffer must have been previously allocated using MbufAlloc2d() with M_ARRAY and 32 + M_FLOAT.	c		e	f	g	a		i		j	k	l	m	n	b
	Transformation matrix object ID in which to copy	Specifies the identifier of the destination transformation matrix object in which to copy the rotation and scale values of the source transformation matrix object. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b

M_TRANSFORMATION_MATRIX	Specifies to set the values of a transformation matrix object using an Aurora Imaging Library array buffer, or to copy a transformation matrix object.	c		e	f	g	a		i		j	k	l	m	n	b
M_IDENTITY_MATRIX	Specifies to copy the identity matrix.	c		e	f	g	a		i		j	k	l	m	n	b
Array buffer identifier in which to copy	Specifies the identifier of the 4x4 Aurora Imaging Library array buffer in which to copy the identity matrix. The Aurora Imaging Library array buffer must have been previously allocated using MbufAlloc2d() with M_ARRAY and 32 + M_FLOAT.	c		e	f	g	a		i		j	k	l	m	n	b
Transformation matrix object ID in which to copy	Specifies the identifier of the destination transformation matrix object in which to copy the identity matrix. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
Array buffer ID to use to define	Specifies the identifier of a 4x4 Aurora Imaging Library array buffer used to set the transformation matrix object. The Aurora Imaging Library array buffer must have been previously allocated using MbufAlloc2d() with M_ARRAY and 32 + M_FLOAT, and must have been initialized.	c		e	f	g	a		i		j	k	l	m	n	b
Transformation matrix object ID	Specifies the identifier of the transformation matrix object. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b

	Transformation matrix object ID from which to copy	Specifies the identifier of a transformation matrix object to copy. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
	Array buffer ID in which to copy	Specifies the identifier of the 4x4 Aurora Imaging Library array buffer in which to copy the values of the transformation matrix object. The Aurora Imaging Library array buffer must have been previously allocated using MbufAlloc2d() with M_ARRAY and 32 + M_FLOAT.	c		e	f	g	a		i		j	k	l	m	n	b
	Transformation matrix object ID in which to copy	Specifies the identifier of the destination transformation matrix object in which to copy the values of the source transformation matrix object. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRANSLATION		Specifies to set only the translation values of a transformation matrix object using an Aurora Imaging Library array buffer, or to copy only the translation values of a transformation matrix object.	c		e	f	g	a		i		j	k	l	m	n	b
	M_IDENTITY_MATRIX	Specifies to copy the translation values of the identity matrix (equivalent to no translation).	c		e	f	g	a		i		j	k	l	m	n	b
	Array buffer ID in which to copy	Specifies the identifier of the 4x4 Aurora Imaging Library array buffer in which to copy the translation values of the identity matrix. The Aurora Imaging Library array buffer must have been previously allocated using MbufAlloc2d() with M_ARRAY and 32 + M_FLOAT.	c		e	f	g	a		i		j	k	l	m	n	b

		Transformation matrix object ID in which to copy	Specifies the identifier of the destination transformation matrix object in which to copy the translation values of the identity matrix. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
	Array buffer ID to use to define		Specifies the identifier of a 4x4 Aurora Imaging Library array buffer used to set the translation values of the transformation matrix object. The Aurora Imaging Library array buffer must have been previously allocated using MbufAlloc2d() with M_ARRAY and 32 + M_FLOAT, and must have been initialized.	c		e	f	g	a		i		j	k	l	m	n	b
		Transformation matrix object ID	Specifies the identifier of the transformation matrix object. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
	Transformation matrix object ID from which to copy		Specifies the identifier of a transformation matrix object from which to copy the translation values. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
		Array buffer ID in which to copy	Specifies the identifier of the 4x4 Aurora Imaging Library array buffer in which to copy the translation values of the transformation matrix object. The Aurora Imaging Library array buffer must have been previously allocated using MbufAlloc2d() with M_ARRAY and 32 + M_FLOAT.	c		e	f	g	a		i		j	k	l	m	n	b

		Transformation matrix object ID in which to copy	Specifies the identifier of the destination transformation matrix object in which to copy the translation values of the source transformation matrix object. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3d11.lib.
DLL	Requires ail11.dll; ail3d11.dll.

M3dgeoCylinder

Synopsis

Define a 3D cylinder geometry object.

Syntax

```
void M3dgeoCylinder(
    AIL_ID Geometry3dgeoId,   //out
    AIL_INT64 CreationMode,   //in
    AIL_DOUBLE XPos1,         //in
    AIL_DOUBLE YPos1,         //in
    AIL_DOUBLE ZPos1,         //in
    AIL_DOUBLE XPos2OrVector, //in
    AIL_DOUBLE YPos2OrVector, //in
    AIL_DOUBLE ZPos2OrVector, //in
    AIL_DOUBLE Radius,        //in
    AIL_DOUBLE Length,        //in
    AIL_INT64 ControlFlag     //in
)
```

Description

This function defines an arbitrary right circular 3D cylinder geometry object. You can translate, rotate, scale, or transform the resulting cylinder, using the 3D image processing module.

You can use the resulting cylinder to, for example, crop a point cloud or perform an arithmetic operation on a depth map using the 3D image processing module, or calculate its distance from each point in a point cloud using the 3D metrology module.

If you want to define a cylinder geometry object from results obtained in a different module, you can use the copy function of that module.

All coordinates are expressed in world units in the working coordinate system.

> Note that if Geometry3dgeoId specifies a previously-defined cylinder, you can leave some of its attributes unchanged, even if that attribute was set using a different creation mode or was modified using the 3D image processing module. To do so, set the corresponding parameter to M_UNCHANGED.

Parameters

Geometry3dgeoId

Specifies the identifier of the 3D geometry object, previously allocated on the required system using M3dgeoAlloc() with M_GEOMETRY.

CreationMode

Specifies how the cylinder is defined.

See the Parameter associations section for possible values that can be specified.

XPos1

Specifies the X-coordinate of a point on the cylinder's central axis.

See the Parameter associations section for possible values that can be specified.

YPos1

Specifies the Y-coordinate of a point on the cylinder's central axis.

See the Parameter associations section for possible values that can be specified.

ZPos1

Specifies the Z-coordinate of a point on the cylinder's central axis.

See the Parameter associations section for possible values that can be specified.

XPos2OrVector

Specifies the X-coordinate of the cylinder's second defining point, or the X-component of the cylinder's central axis vector.

See the Parameter associations section for possible values that can be specified.

YPos2OrVector

Specifies the Y-coordinate of the cylinder's second defining point, or the Y-component of the cylinder's central axis vector.

See the Parameter associations section for possible values that can be specified.

ZPos2OrVector

Specifies the Z-coordinate of the cylinder's second defining point, or the Z-component of the cylinder's central axis vector.

See the Parameter associations section for possible values that can be specified.

Radius INQ

Specifies the cylinder's radius.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the cylinder's radius																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the cylinder's radius.	c		e	f	g	a		i		j	k	l	m	n	b

Length

Specifies to override the default cylinder length.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying to override the default cylinder length																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default length defined by the creation mode.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies an infinite cylinder.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies to override the cylinder's length with a specific value.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag INQ

Specifies whether the resulting cylinder has bases. Cylinders with Length set to M_INFINITE cannot have bases.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying if the cylinder has bases																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default value. For a finite cylinder, this value is the same as M_WITH_BASES. For an infinite cylinder, this value is the same as M_WITHOUT_BASES.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
M_WITH_BASES	Specifies that the resulting cylinder has a circular base at each end. This value is only supported for finite cylinders.	c		e	f	g	a		i		j	k	l	m	n	b
M_WITHOUT_BASES	Specifies that the resulting cylinder does not have bases.	c		e	f	g	a		i		j	k	l	m	n	b

Parameter associations

The table below lists possible values for the **CreationMode**, **XPos1**, **YPos1**, **ZPos1**, **XPos2OrVector**, **YPos2OrVector**, and **ZPos2OrVector** parameters.

- For specifying the cylinder

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the cylinder INQ																		
CreationMode	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
XPos1																
YPos1																
ZPos1																
XPos2OrVector																
YPos2OrVector																
ZPos2OrVector																
M_POINT_AND_VECTOR	Defines the cylinder using a point on the cylinder's central axis and a nonzero vector defining the central axis direction. By default, the length of the cylinder is finite and is set to the vector's magnitude. The point is at the center of the cylinder's first circular base. Note that if you set one of the vector's components to M_UNCHANGED, you must also set the other two components to M_UNCHANGED.															
XPos1	Specifies the X-coordinate of the point on the cylinder's central axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-coordinate of the first point on the cylinder's central axis.	c		e	f	g	a		i		j	k	l	m	n	b
YPos1	Specifies the Y-coordinate of the point on the cylinder's central axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-coordinate of the first point on the cylinder's central axis.	c		e	f	g	a		i		j	k	l	m	n	b
ZPos1	Specifies the Z-coordinate of the point on the cylinder's central axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b

Value	Specifies the Z-coordinate of the first point on the cylinder's central axis.	c		e	f	g	a		i		j	k	l	m	n	b
XPos2OrVector	Specifies the X-component of the vector defining the central axis direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-component of the vector defining the central axis direction.	c		e	f	g	a		i		j	k	l	m	n	b
YPos2OrVector	Specifies the Y-component of the vector defining the central axis direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-component of the vector defining the central axis direction.	c		e	f	g	a		i		j	k	l	m	n	b
ZPos2OrVector	Specifies the Z-component of the vector defining the central axis direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Z-component of the vector defining the central axis direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_TWO_POINTS	Defines the cylinder using any two non-identical points on the cylinder's central axis. By default, the length of the cylinder is finite and is set to the distance between the two specified points. The first specified point is at the center of the cylinder's first circular base and the second specified point is at the center of the cylinder's second circular base.	c		e	f	g	a		i		j	k	l	m	n	b
XPos1	Specifies the X-coordinate of the first point on the cylinder's central axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-coordinate of the first point on the cylinder's central axis.	c		e	f	g	a		i		j	k	l	m	n	b

	YPos1	Specifies the Y-coordinate of the first point on the cylinder's central axis.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the Y-coordinate of the first point on the cylinder's central axis.	c		e	f	g	a		i		j	k	l	m	n	b
	ZPos1	Specifies the Z-coordinate of the first point on the cylinder's central axis.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the Z-coordinate of the first point on the cylinder's central axis.	c		e	f	g	a		i		j	k	l	m	n	b
	XPos2OrVector	Specifies the X-coordinate of the second point on the cylinder's central axis.	c		e	f	g	a		i		j	k	l	m	n	b
	YPos2OrVector	Specifies the Y-coordinate of the second point on the cylinder's central axis.	c		e	f	g	a		i		j	k	l	m	n	b
	ZPos2OrVector	Specifies the Z-coordinate of the second point on the cylinder's central axis.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3d11.lib.
DLL	Requires ail11.dll; ail3d11.dll.

M3dgeoDraw3d

Synopsis

Draw the geometry, defined in the specified 3D geometry object, into a 3D graphics list.

Syntax

```
AIL_INT64 M3dgeoDraw3d(
    AIL_ID OperationDraw3dContext3dgeoId, //in
    AIL_ID SrcGeometry3dgeoId,            //in
    AIL_ID DstList3dgraId,                //out
    AIL_INT64 DstParentLabel,             //in
    AIL_INT64 ControlFlag                 //in
)
```

Description

This function draws a geometry, defined in the specified 3D geometry object, into the destination 3D graphics list.

Parameters

OperationDraw3dContext3dgeoId

Reserved for future expansion and must be set to **M_DEFAULT**.

SrcGeometry3dgeoId

Specifies the identifier of the 3D geometry object to draw into the 3D graphics list. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and must have been successfully defined. Supported 3D geometries include box, cylinder, line, plane, point, and sphere.

For geometries with infinite size, such as infinite lines, planes, and cylinders, the 3D graphics list's clipping box is used to determine the position and size of the 3D graphic. The size and position are only set when calling **M3dgeoDraw3d**(); they do not update if the 3D graphics list's clipping box is changed afterward.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the geometry object identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_XY_PLANE	Specifies the XY (Z=0) plane.	c		e	f	g	a		i		j	k	l	m	n	b
3D geometry object identifier	Specifies the identifier of a 3D geometry object. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY.	c		e	f	g	a		i		j	k	l	m	n	b

DstList3dgraId

Specifies the identifier of the 3D graphics list in which to draw. The 3D graphics list must have been previously allocated using M3dgraAlloc(), or you can specify the identifier of the 3D display's internal

graphics list (inquired using M3ddispInquire() with M_3D_GRAPHIC_LIST_ID).

DstParentLabel

Specifies the label of the 3D graphic in the 3D graphics list to be used as the geometry's parent.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the parent label																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_ROOT_NODE.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROOT_NODE	Specifies the top-most node of the 3D graphics list.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the label of the 3D graphic in the 3D graphics list. Label 0 is the 3D graphics list's root node.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Return value

Returns the label of the 3D graphic added to the 3D graphics list.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3d11.lib.
DLL	Requires ail11.dll; ail3d11.dll.

M3dgeoEvalCurve

Synopsis

Evaluate a curve and a list of partial coordinates and find the missing coordinates for points that lie on the curve.

Syntax

```
AIL_INT M3dgeoEvalCurve(
    AIL_ID Geometry3dgeoId,             //in
    AIL_INT64 Operation,                //in
    AIL_INT NumPoints,                  //in
    const AIL_DOUBLE *SrcCoordArrayPtr, //in
    AIL_DOUBLE *Dst1CoordArrayPtr,      //out
    AIL_DOUBLE *Dst2CoordArrayPtr,      //out
    AIL_INT64 ControlFlag               //in
)
```

Description

This function evaluates a curve (such as a 3D line) and a list of partial coordinates and calculates the missing coordinates for points that lie on the curve. You must provide a source array of coordinates in one dimension (for example, X-coordinates). **M3dgeoEvalCurve**() calculates the coordinates for the other two dimensions (for example, the missing Y- and Z-coordinates). Only coordinates that lie on the given source curve are calculated.

M3dgeoEvalCurve() supports 3D line geometries only. Note that a 3D line has a dimensionality of one, which means that, given a single coordinate for a point that lies on the line, it is possible to find the other two coordinates. You can inquire a 3D geometry's dimensionality using M3dgeoInquire() with M_DIMENSION.

For each provided source coordinate, there are two possible cases:

- Exactly one point with the specified coordinate exists on the 3D line. The missing coordinates are calculated and written to the corresponding entries in the destination arrays.
- No singular point exists. This occurs when no point with the specified coordinate exists on the 3D line, or when infinite remaining coordinate pairs are possible. If no singular point exists, or if infinite points are possible, **M_INVALID_POINT** (or **M_INVALID_POINT_FLOAT**, in the case of an array of type *AIL_FLOAT*) is written to the corresponding entries in the destination arrays. For example, no coordinates are calculated for any source coordinate that is beyond the limits of a finite line. Also note that if the source 3D line is parallel to an axis plane (for example, the XY-plane), providing any coordinate on the axis that is perpendicular to the plane returns **M_INVALID_POINT**.

> Note that presented above is an example prototype of this function. You can call M3dgeoEvalCurve() with the same prototype as any of the type-specific functions below, without using the function's type-specific name; that is, use M3dgeoEvalCurve() and not, for example, M3dgeoEvalCurveFloat(). The data is automatically cast into the appropriate type. This is true except when using a C compiler. When using a C compiler, you must always call this function with its type-specific name (you cannot use the standard name of this function).

Parameters

Geometry3dgeoId

Specifies the identifier of the 3D geometry object used to evaluate 3D points. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and successfully defined as a line.

Operation

Specifies the coordinates to evaluate (establish).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the coordinates to evaluate (establish)																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_EVAL_XY +	Specifies to establish the X- and Y-coordinates of the source 3D geometry, for each provided Z-coordinate. Results are written to the corresponding entry in the destination arrays. If destination coordinates are not packed, the first and second destination arrays hold the X- and Y-coordinates, respectively.	c		e	f	g	a		i		j	k	l	m	n	b
M_EVAL_XZ +	Specifies to establish the X- and Z-coordinates of the source 3D geometry, for each provided Y-coordinate. Results are written to the corresponding entry in the destination arrays. If destination coordinates are not packed, the first and second destination arrays hold the X- and Z-coordinates, respectively.	c		e	f	g	a		i		j	k	l	m	n	b
M_EVAL_YZ +	Specifies to establish the Y- and Z-coordinates of the source 3D geometry, for each provided X-coordinate. Results are written to the corresponding entry in the destination arrays. If destination coordinates are not packed, the first and second destination arrays hold the Y- and Z-coordinates, respectively.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_EVAL_XY; M_EVAL_XZ; M_EVAL_YZ.

You can add one of the following values to the above-mentioned values to specify to use packed sets of X-, Y-, and Z-coordinates.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the storage format																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_PACKED	Same as specifying both M_PACKED_SRC and M_PACKED_DST.	c		e	f	g	a		i		j	k	l	m	n	b

M_PACKED_DST	Specifies to store the point data in a packed format in the Dst1CoordArrayPtr destination array; that is, the coordinates will be stored together (XYZ XYZ XYZ...). Dst2CoordArrayPtr must be set to **M_NULL**. > Note that all three coordinates will be written to the first destination array.	c		e	f	g	a		i		j	k	l	m	n	b
M_PACKED_SRC	Specifies that source points are provided in a packed format in the source array; that is, the coordinates are stored together (XYZ XYZ XYZ...). > Note that, if packed, not all coordinates are considered. For example, if you specify an M_EVAL_XY operation, the function uses only the source Z-coordinates in the evaluation.	c		e	f	g	a		i		j	k	l	m	n	b

NumPoints

Specifies the number of points in the arrays.

> When using a standard vector (std::vector) overload function in C++, you can pass **M_DEFAULT** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the SrcCoordArrayPtr parameter.

SrcCoordArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(NumPoints)
- M_NULL

Specifies the address of the array that contains the source coordinates, or, when M_PACKED or M_PACKED_SRC is specified, the address of the array that contains a packed set of source X-, Y-, and Z-coordinates.

Dst1CoordArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(NumPoints)
- M_NULL

Specifies the address of the array in which to store the first set of destination coordinates, or, when M_PACKED or M_PACKED_DST is specified, the address of the array that contains a packed set of destination X-, Y-, and Z-coordinates.

Dst2CoordArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(NumPoints)
- M_NULL

Specifies the address of the array in which to store the second set of destination coordinates. This parameter must be set to **M_NULL** when M_PACKED or M_PACKED_DST is specified.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Return value

The returned value is the number of valid points written to the destination array(s); that is, the number of points whose coordinates match the 3D line geometry and are not set to **M_INVALID_POINT** or **M_INVALID_POINT_FLOAT**.

Type-specific versions of the function when using a C compiler

```
AIL_INT M3dgeoEvalCurveDouble (AIL_ID Geometry3dgeoId, AIL_INT64 Operation, AIL_INT NumPoints, const AIL_DOUBLE *SrcCoordArrayPtr, AIL_DOUBLE *Dst1CoordArrayPtr, AIL_DOUBLE *Dst2CoordArrayPtr, AIL_INT64 ControlFlag)
```

Details

Parameters

Geometry3dgeoId

See Geometry3dgeoId of the main function for a description.

Operation

See Operation of the main function for a description.

NumPoints

See NumPoints of the main function for a description.

SrcCoordArrayPtr

See SrcCoordArrayPtr of the main function for a description.

Dst1CoordArrayPtr

See Dst1CoordArrayPtr of the main function for a description.

Dst2CoordArrayPtr

See Dst2CoordArrayPtr of the main function for a description.

ControlFlag

See ControlFlag of the main function for a description.

```
AIL_INT M3dgeoEvalCurveFloat (AIL_ID Geometry3dgeoId, AIL_INT64 Operation, AIL_INT NumPoints, const AIL_FLOAT *SrcCoordArrayPtr, AIL_FLOAT *Dst1CoordArrayPtr, AIL_FLOAT *Dst2CoordArrayPtr, AIL_INT64 ControlFlag)
```

Details

Parameters

Geometry3dgeoId

See Geometry3dgeoId of the main function for a description.

Operation

See Operation of the main function for a description.

NumPoints

See NumPoints of the main function for a description.

SrcCoordArrayPtr

See SrcCoordArrayPtr of the main function for a description.

Dst1CoordArrayPtr

See Dst1CoordArrayPtr of the main function for a description.

Dst2CoordArrayPtr

See Dst2CoordArrayPtr of the main function for a description.

ControlFlag

See ControlFlag of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3d11.lib.
DLL	Requires ail11.dll; ail3d11.dll.

M3dgeoEvalSurface

Synopsis

Evaluate a surface and a list of partial coordinates and find the missing coordinates for points that lie on the surface.

Syntax

```
AIL_INT M3dgeoEvalSurface(
    AIL_ID Geometry3dgeoId,              //in
    AIL_INT64 Operation,                 //in
    AIL_INT NumPoints,                   //in
    const AIL_DOUBLE *Src1CoordArrayPtr, //in
    const AIL_DOUBLE *Src2CoordArrayPtr, //in
    AIL_DOUBLE *DstCoordArrayPtr,        //out
    AIL_INT64 ControlFlag                //in
)
```

Description

This function evaluates a surface (such as a 3D plane or sphere) and a list of partial coordinates and calculates the missing coordinates for points that lie on the surface. You must provide source coordinates in two dimensions (for example, X- and Y-coordinates). **M3dgeoEvalSurface**() calculates the coordinates for the third dimension (for example, the missing Z-coordinates). Only coordinates that lie on the given geometry's surface are calculated.

M3dgeoEvalSurface() supports 3D plane and 3D sphere geometries only. Note that these geometries have a dimensionality of two, which means that, given two coordinates for a point that lies on the surface, it is possible to find the third coordinate. You can inquire a 3D geometry's dimensionality using M3dgeoInquire() with M_DIMENSION.

For each provided source coordinate pair, there are three possible cases:

- Exactly one point with the specified coordinates exists on the 3D geometry's surface. The missing coordinate is calculated and written to the corresponding entry in the destination array.
- Exactly two points with the specified coordinates exist on the 3D geometry's surface. This can occur with a 3D sphere geometry. In this case, you can specify to write the largest or smallest calculated coordinate to the corresponding entry in the destination array (with M_MAX_VALUE or M_MIN_VALUE, respectively).
- No point exists. This can occur when the provided coordinates do not match the source 3D geometry (for example, when the coordinates match a sphere with a different radius, or when the coordinates match a parallel plane that does not intersect with the provided plane). If no point exists, the value **M_INVALID_POINT** (or **M_INVALID_POINT_FLOAT**, in the case of an array of type *AIL_FLOAT*) is written to the corresponding entry in the destination array.

> Note that presented above is an example prototype of this function. You can call M3dgeoEvalSurface() with the same prototype as any of the type-specific functions below, without using the function's type-specific name; that is, use M3dgeoEvalSurface() and not, for example, M3dgeoEvalSurfaceFloat(). The data is automatically cast into the appropriate type. This is true except when using a C compiler. When using a C compiler, you must always call this function with its type-specific name (you cannot use the

> standard name of this function).

Parameters

Geometry3dgeoId

Specifies the identifier of the 3D geometry object used to evaluate 3D points. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and successfully defined as a plane or sphere.

Operation

Specifies the coordinate to evaluate (establish).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the coordinate to evaluate (establish).																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_EVAL_X +	Specifies to establish the X-coordinate of the source 3D geometry, for each provided pair of Y- and Z-coordinates. Results are written to the corresponding entry in the destination array. If destination coordinates are not packed, the destination array holds the X-coordinates.	c		e	f	g	a		i		j	k	l	m	n	b
M_EVAL_Y +	Specifies to establish the Y-coordinate of the source 3D geometry, for each provided pair of X- and Z-coordinates. Results are written to the corresponding entry in the destination array. If destination coordinates are not packed, the destination array holds the Y-coordinates.	c		e	f	g	a		i		j	k	l	m	n	b
M_EVAL_Z +	Specifies to establish the Z-coordinate of the source 3D geometry, for each provided pair of X- and Y-coordinates. Results are written to the corresponding entry in the destination array. If destination coordinates are not packed, the destination array holds the Z-coordinates.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_EVAL_X; M_EVAL_Y; M_EVAL_Z.

You can add one of the following values to the above-mentioned values to specify which coordinate to store in the destination array when multiple coordinates are calculated.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying which evaluated point will be returned																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_MAX_VALUE	Stores the largest calculated coordinate for each entry in the destination array. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

M_MIN_VALUE	Stores the smallest calculated coordinate for each entry in the destination array.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_EVAL_X; M_EVAL_Y; M_EVAL_Z.

You can add one of the following values to the above-mentioned values to specify to use packed sets of X-, Y-, and Z-coordinates.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the storage format																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_PACKED	Same as specifying both M_PACKED_SRC and M_PACKED_DST.	c		e	f	g	a		i		j	k	l	m	n	b
M_PACKED_DST	Specifies to store the point data in a packed format in the destination array; that is, the coordinates will be stored together (XYZ XYZ XYZ...). > Note that all three coordinates will be written to the destination array.	c		e	f	g	a		i		j	k	l	m	n	b
M_PACKED_SRC	Specifies that source points are provided in a packed format in the Src1CoordArrayPtr source array; that is, the coordinates are stored together (XYZ XYZ XYZ...). Src2CoordArrayPtr must be set to **M_NULL**. > Note that, if packed, not all coordinates are considered. For example, if you specify an M_EVAL_X operation, the function uses only the source Y- and Z-coordinates in the evaluation.	c		e	f	g	a		i		j	k	l	m	n	b

NumPoints

Specifies the number of points in the arrays.

> When using a standard vector (std::vector) overload function in C++, you can pass **M_DEFAULT** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the Src1CoordArrayPtr or Src2CoordArrayPtr parameter.

Src1CoordArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(NumPoints)
- M_NULL

Specifies the address of the array that contains the first source coordinates, or, when M_PACKED or

M_PACKED_SRC is specified, the address of the array that contains a packed set of source X-, Y-, and Z-coordinates.

Src2CoordArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(NumPoints)
- M_NULL

Specifies the address of the array that contains the second source coordinates. This parameter must be set to **M_NULL** when M_PACKED or M_PACKED_SRC is specified.

DstCoordArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(NumPoints)
- M_NULL

Specifies the address of the array in which to store the set of destination coordinates, or, when M_PACKED or M_PACKED_DST is specified, the address of the array that contains a packed set of destination X-, Y-, and Z-coordinates.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Return value

The returned value is the number of valid points written to the destination array(s); that is, the number of points whose coordinates match the 3D geometry and are not set to **M_INVALID_POINT** or **M_INVALID_POINT_FLOAT**.

Type-specific versions of the function when using a C compiler

```
AIL_INT M3dgeoEvalSurfaceDouble (AIL_ID Geometry3dgeoId, AIL_INT64 Operation, AIL_INT NumPoints, const AIL_DOUBLE *Src1CoordArrayPtr, const AIL_DOUBLE *Src2CoordArrayPtr, AIL_DOUBLE *DstCoordArrayPtr, AIL_INT64 ControlFlag)
```

Details

Parameters

Geometry3dgeoId

See Geometry3dgeoId of the main function for a description.

Operation

See Operation of the main function for a description.

NumPoints

See NumPoints of the main function for a description.

Src1CoordArrayPtr

See Src1CoordArrayPtr of the main function for a description.

Src2CoordArrayPtr

See Src2CoordArrayPtr of the main function for a description.

DstCoordArrayPtr

See DstCoordArrayPtr of the main function for a description.

ControlFlag

See ControlFlag of the main function for a description.

```
AIL_INT M3dgeoEvalSurfaceFloat (AIL_ID Geometry3dgeoId, AIL_INT64 Operation, AIL_INT NumPoints, const AIL_FLOAT *Src1CoordArrayPtr, const AIL_FLOAT *Src2CoordArrayPtr, AIL_FLOAT *DstCoordArrayPtr, AIL_INT64 ControlFlag)
```

Details

Parameters

Geometry3dgeoId

See Geometry3dgeoId of the main function for a description.

Operation

See Operation of the main function for a description.

NumPoints

See NumPoints of the main function for a description.

Src1CoordArrayPtr

See Src1CoordArrayPtr of the main function for a description.

Src2CoordArrayPtr

See Src2CoordArrayPtr of the main function for a description.

DstCoordArrayPtr

See DstCoordArrayPtr of the main function for a description.

ControlFlag

See ControlFlag of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3d11.lib.
DLL	Requires ail11.dll; ail3d11.dll.

M3dgeoFree

Synopsis

Free a 3D geometry object or transformation matrix object.

Syntax

```
void M3dgeoFree(
    AIL_ID GeometryOrMatrix3dgeoId //in
)
```

Description

This function deletes the specified 3D geometry object or transformation matrix object and releases any memory associated with it.

All 3D geometry objects or transformation matrix objects allocated on a particular system must be freed before the system can be freed, unless M_UNIQUE_ID was specified during allocation.

Parameter

GeometryOrMatrix3dgeoId

Specifies the identifier of the 3D geometry object or transformation matrix object to free. These must have been successfully allocated with M3dgeoAlloc() prior to calling this function.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3d11.lib.
DLL	Requires ail11.dll; ail3d11.dll.

M3dgeoInquire

Synopsis

Inquire about a 3D geometry object setting or transformation matrix object setting.

Syntax

```
AIL_DOUBLE M3dgeoInquire(
    AIL_ID GeometryOrMatrix3dgeoId, //in
    AIL_INT64 InquireType,          //in
    void *UserVarPtr                //out
)
```

Description

This function inquires about a specified setting of a 3D geometry object or transformation matrix object.

> Note that if the specified 3D geometry object was transformed (for example, using the 3D image processing module), this function returns the modified coordinates and dimensions. In this case, the coordinates and dimensions returned by **M3dgeoInquire**() might not be the same initial coordinates and dimensions used to define the 3D geometry object.

Parameters

GeometryOrMatrix3dgeoId

Specifies the identifier of the 3D geometry object or transformation matrix object about which to inquire information. The object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY or M_TRANSFORMATION_MATRIX.

See the Parameter associations section for possible values that can be specified.

InquireType

Specifies the setting to inquire.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- AIL_DOUBLE
- AIL_FLOAT
- AIL_INT
- AIL_INT32
- AIL_INT64

Specifies the address in which to write the requested information. Since the **M3dgeoInquire**() function

also returns the requested information, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **GeometryOrMatrix3dgeoId** and **InquireType** parameters and possible values returned to the **UserVarPtr** parameter.

- For inquiring about any 3D geometry object
- For inquiring about a specific type of 3D geometry object
- For inquiring about any transformation matrix object

For any 3D geometry object, the GeometryOrMatrix3dgeoId and InquireType parameters can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about any 3D geometry object																	
GeometryOrMatrix3dgeoId / InquireType / UserVarPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
3D geometry object ID	Specifies a 3D geometry object, allocated using M3dgeoAlloc() with M_GEOMETRY.															
M_DIMENSION +	Inquires the dimensionality of the 3D geometry. The dimensionality specifies, for a point on the geometry, the number of coordinates you must have to determine all three coordinates. For example, for a 3D line (dimensionality of 1), if you have the X-coordinate of a point on the line, you can find the corresponding Y- and Z-coordinates. Similarly, for a plane (dimensionality of 2), if you have the X- and Y-coordinates for a point on the plane, you can find the corresponding Z-coordinate. Use M3dgeoEvalCurve() to find missing coordinates for a 3D line; use M3dgeoEvalSurface() to find missing coordinates for a 3D plane or sphere.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 2	Specifies the dimensionality of the 3D geometry.	c		e	f	g	a		i		j	k	l	m	n	b
M_GEOMETRY_TYPE +	Inquires the 3D geometry type.	c		e	f	g	a		i		j	k	l	m	n	b

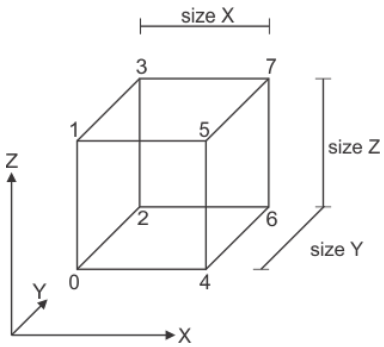
		UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
		M_BOX	Specifies a box.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CYLINDER	Specifies a cylinder.	c		e	f	g	a		i		j	k	l	m	n	b
		M_LINE	Specifies a line.	c		e	f	g	a		i		j	k	l	m	n	b
		M_NOT_INITIALIZED	Specifies that a geometry has not been defined for the 3D geometry object.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PLANE	Specifies a plane.	c		e	f	g	a		i		j	k	l	m	n	b
		M_POINT	Specifies a point.	c		e	f	g	a		i		j	k	l	m	n	b
		M_SPHERE	Specifies a sphere.	c		e	f	g	a		i		j	k	l	m	n	b

For a 3D geometry object of a specific type, the GeometryOrMatrix3dgeoId and InquireType parameters can be set to one of the following.

> Note that for any 3D geometry object, you can use any inquire type available for its respective geometry type, regardless of the creation mode used to define the geometry.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about a specific type of 3D geometry object																		
GeometryOrMatrix3dgeoId			Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	InquireType																	
		UserVarPtr - Possible values returned																

3D box geometry object ID	Specifies a 3D box geometry object, allocated using M3dgeoAlloc() with M_GEOMETRY and successfully defined as a box. Note that a box's corners are assigned the following indices upon initially defining the box, based on the corner's position with respect to the axes of the working coordinate system. The index of a box corner does not change based on the orientation of the box.  													
M_CORNER_X(AIL_INT CornerIndex) +	Inquires the X-coordinate of the specified box corner.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters														
	CornerIndex Specifies the index of the box corner.														
	0 <= *Value* <= 7 Specifies the box corner.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-coordinate of the specified box corner, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_CORNER_Y(AIL_INT CornerIndex) +	Inquires the Y-coordinate of the specified box corner.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters														

		CornerIndex Specifies the index of the box corner.														
		0 <= *Value* <= 7 — Specifies the box corner.	c	e	f	g	a	i	j	k	l	m	n	b		
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c	e	f	g	a	i	j	k	l	m	n	b		
	Value	Specifies the Y-coordinate of the specified box corner, expressed in the working coordinate system.	c	e	f	g	a	i	j	k	l	m	n	b		
	M_CORNER_Z(AIL_INT CornerIndex) +	Inquires the Z-coordinate of the specified box corner.	c	e	f	g	a	i	j	k	l	m	n	b		
		Parameters														
		CornerIndex Specifies the index of the box corner.														
		0 <= *Value* <= 7 — Specifies the box corner.	c	e	f	g	a	i	j	k	l	m	n	b		
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c	e	f	g	a	i	j	k	l	m	n	b		
	Value	Specifies the Z-coordinate of the specified box corner, expressed in the working coordinate system.	c	e	f	g	a	i	j	k	l	m	n	b		
	M_FACE_CORNERS_X(AIL_INT FaceIndex) +	Inquires the X-coordinates of each corner of the specified box face, in their index order.	c	e	f	g	a	i	j	k	l	m	n	b		
		Parameters														
		FaceIndex Specifies the index of the box face.														
		0 <= *Value* <= 5 — Specifies the box face.	c	e	f	g	a	i	j	k	l	m	n	b		

UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-coordinates of the corners of the specified box face, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_FACE_CORNERS_Y(AIL_INT FaceIndex) +	Inquires the Y-coordinates of each corner of the specified box face, in their index order.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	FaceIndex Specifies the index of the box face.															
	0 <= *Value* <= 5 Specifies the box face.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-coordinates of the corners of the specified box face, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_FACE_CORNERS_Z(AIL_INT FaceIndex) +	Inquires the Z-coordinates of each corner of the specified box face, in their index order.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	FaceIndex Specifies the index of the box face.															
	0 <= *Value* <= 5 Specifies the box face.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

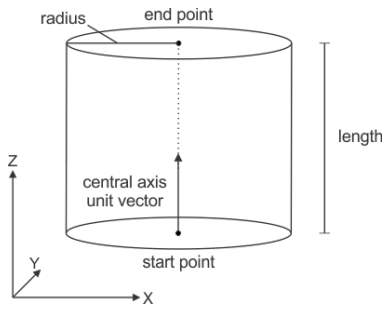
		Value	Specifies the Z-coordinates of the corners of the specified box face, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AXIS_ALIGNED +		Inquires whether the box is axis-aligned with the working coordinate system. Note that this inquire type returns the opposite of the M_ROTATED inquire type, except when an axis-aligned box is rotated by 180 degrees; in this case, M_AXIS_ALIGNED and M_ROTATED both return M_TRUE.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
		M_FALSE	Specifies that the box is not axis-aligned; a rotation (other than a 180 degree rotation) has been applied.	c		e	f	g	a		i		j	k	l	m	n	b
		M_TRUE	Specifies that the box is axis-aligned; no rotation (or a 180 degree rotation) has been applied.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CENTER_X +		Inquires the X-coordinate of the box's center point. SET	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the X-coordinate of the box's center point, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CENTER_Y +		Inquires the Y-coordinate of the box's center point. SET	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the Y-coordinate of the box's center point, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CENTER_Z +		Inquires the Z-coordinate of the box's center point. SET	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

		Value	Specifies the Z-coordinate of the box's center point, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CORNER_X_ALL +		Inquires the X-coordinate of each of the box's 8 corners, in their index order.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the X-component the box's corner, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CORNER_Y_ALL +		Inquires the Y-coordinate of each of the box's 8 corners, in their index order.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the Y-component the box's corner, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CORNER_Z_ALL +		Inquires the Z-coordinate of each of the box's 8 corners, in their index order.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the Z-component the box's corner, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FACE_*n*_CORNERS_X +		Inquires the X-coordinates of each corner of box face *n*, in their index order, where *n* is the box face index.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the X-coordinates of the corners of the box face assigned with index *n*, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b

M_FACE_*n*_CORNERS_Y +	Inquires the Y-coordinates of each corner of box face *n*, in their index order, where *n* is the box face index.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-coordinates of the corners of the box face assigned with index *n*, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_FACE_*n*_CORNERS_Z +	Inquires the Z-coordinates of each corner of box face *n*, in their index order, where *n* is the box face index.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Z-coordinates of the corners of the box face assigned with index *n*, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATED +	Inquires whether the box has been rotated. This inquire type returns true when the box's rotation matrix is not the identity matrix. This can occur even if you did not use, for example, M3dimRotate() or M3dimMatrixTransform() to explicitly rotate the box. If you copied the box from a statistics 3D image processing result buffer (after calculating a semi-oriented box) or a 3D model finder result buffer, it can have a rotation matrix other than the identity matrix. Note that this inquire type returns the opposite of the M_AXIS_ALIGNED inquire type, except when an axis-aligned box is rotated by 180 degrees; in this case, M_AXIS_ALIGNED and M_ROTATED both return M_TRUE.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the box is not rotated; its rotation matrix is the identity matrix.	c		e	f	g	a		i		j	k	l	m	n	b

	M_TRUE	Specifies that the box is rotated; its rotation matrix is not the identity matrix.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_X +		Inquires the box's length along the X-axis, ignoring the box's rotation. The length is returned as if the box is axis-aligned. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the length of the unrotated box, in world units, along the X-axis of the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_Y +		Inquires the box's length along the Y-axis, ignoring the box's rotation. The length is returned as if the box is axis-aligned. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the length of the unrotated box, in world units, along the Y-axis of the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_Z +		Inquires the box's length along the Z-axis, ignoring the box's rotation. The length is returned as if the box is axis-aligned. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the length of the unrotated box, in world units, along the Z-axis of the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNROTATED_MAX_X +		Inquires the box's maximum X-coordinate, ignoring the box's rotation. The coordinate is returned as if the box is axis-aligned.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the box's maximum X-coordinate, ignoring the box's rotation, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNROTATED_MAX_Y +	Inquires the box's maximum Y-coordinate, ignoring the box's rotation. The coordinate is returned as if the box is axis-aligned.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the box's maximum Y-coordinate, ignoring the box's rotation, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNROTATED_MAX_Z +	Inquires the box's maximum Z-coordinate, ignoring the box's rotation. The coordinate is returned as if the box is axis-aligned.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the box's maximum Z-coordinate, ignoring the box's rotation, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNROTATED_MIN_X +	Inquires the box's minimum X-coordinate, ignoring the box's rotation. The coordinate is returned as if the box is axis-aligned.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the box's minimum X-coordinate, ignoring the box's rotation, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNROTATED_MIN_Y +	Inquires the box's minimum Y-coordinate, ignoring the box's rotation. The coordinate is returned as if the box is axis-aligned.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the box's minimum Y-coordinate, ignoring the box's rotation, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNROTATED_MIN_Z +	Inquires the box's minimum Z-coordinate, ignoring the box's rotation. The coordinate is returned as if the box is axis-aligned.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the box's minimum Z-coordinate, ignoring the box's rotation, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
3D cylinder geometry object ID	Specifies a 3D cylinder geometry object, allocated using M3dgeoAlloc() with M_GEOMETRY and successfully defined as a cylinder. 	c		e	f	g	a		i		j	k	l	m	n	b
M_AXIS_X +	Inquires the X-component of the cylinder's central axis unit vector. This vector does not reflect the cylinder's length, regardless of whether a vector was used to define the cylinder's length.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
-1.0 >= *Value* >= 1.0	Specifies the X-component of the cylinder's central axis unit vector, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_AXIS_Y +	Inquires the Y-component of the cylinder's central axis unit vector. This vector does not reflect the cylinder's length, regardless of whether a vector was used to define the cylinder's length.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	-1.0 >= *Value* >= 1.0	Specifies the Y-component of the cylinder's central axis unit vector, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_AXIS_Z +		Inquires the Z-component of the cylinder's central axis unit vector. This vector does not reflect the cylinder's length, regardless of whether a vector was used to define the cylinder's length.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	-1.0 >= *Value* >= 1.0	Specifies the Z-component of the cylinder's central axis unit vector, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_BASES +		Inquires whether the cylinder has bases. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_WITH_BASES	Specifies that the cylinder has bases.	c		e	f	g	a		i		j	k	l	m	n	b
	M_WITHOUT_BASES	Specifies that the cylinder does not have bases.	c		e	f	g	a		i		j	k	l	m	n	b
M_CENTER_X +		Inquires the X-coordinate of the center point on the cylinder's central axis. This inquire type is only available if the cylinder's length is not M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the X-coordinate of the center point on the cylinder's central axis, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_CENTER_Y +		Inquires the Y-coordinate of the center point on the cylinder's central axis. This inquire type is only available if the cylinder's length is not M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	Value	Specifies the Y-coordinate of the center point on the cylinder's central axis, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_CENTER_Z +		Inquires the Z-coordinate of the center point on the cylinder's central axis. This inquire type is only available if the cylinder's length is not M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the Z-coordinate of the center point on the cylinder's central axis, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_END_POINT_X +		Inquires the X-coordinate of the cylinder's end point, positioned at the center of the cylinder's second circular base. This inquire type is only available if the cylinder's length is not M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the X-coordinate of the cylinder's end point, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_END_POINT_Y +		Inquires the Y-coordinate of the cylinder's end point, positioned at the center of the cylinder's second circular base. This inquire type is only available if the cylinder's length is not M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the Y-coordinate of the cylinder's end point, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_END_POINT_Z +		Inquires the Z-coordinate of the cylinder's end point, positioned at the center of the cylinder's second circular base. This inquire type is only available if the cylinder's length is not M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b

		UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the Z-coordinate of the cylinder's end point, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LENGTH +		Inquires the cylinder's length.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
		M_INFINITE	Specifies that the cylinder is infinite.	c		e	f	g	a		i		j	k	l	m	n	b
		Value >= 0.0	Specifies the cylinder's length in world units.	c		e	f	g	a		i		j	k	l	m	n	b
	M_RADIUS +		Inquires the cylinder's radius. SET	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
		Value >= 0.0	Specifies the cylinder's radius in world units.	c		e	f	g	a		i		j	k	l	m	n	b
	M_START_POINT_X +		Inquires the X-coordinate of the cylinder's start point on the cylinder's central axis. When the cylinder's length is infinite, this returns the X-coordinate of the first point used to define the cylinder. SET	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the X-coordinate of the first point on the central axis used to define the cylinder, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_START_POINT_Y +		Inquires the Y-coordinate of the cylinder's start point on the cylinder's central axis. When the cylinder's length is infinite, this returns the Y-coordinate of the first point used to define the cylinder. SET	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

		Value	Specifies the Y-coordinate of the first point on the central axis used to define the cylinder, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_START_POINT_Z +		Inquires the Z-coordinate of the cylinder's start point on the cylinder's central axis. When the cylinder's length is infinite, this returns the Z-coordinate of the first point used to define the cylinder. SET	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the Z-coordinate of the first point on the central axis used to define the cylinder, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
3D line geometry object ID			Specifies a 3D line geometry object, allocated using M3dgeoAlloc() with M_GEOMETRY and successfully defined as a line.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AXIS_X +		Inquires the X-component of the line's direction unit vector. This vector does not reflect the line's length, regardless of whether a vector was used to define the line's length.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
		-1.0 >= *Value* >= 1.0	Specifies the X-component of the line's direction unit vector, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AXIS_Y +		Inquires the Y-component of the line's direction unit vector. This vector does not reflect the line's length, regardless of whether a vector was used to define the line's length.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
		-1.0 >= *Value* >= 1.0	Specifies the Y-component of the line's direction unit vector, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b

M_AXIS_Z +	Inquires the Z-component of the line's direction unit vector. This vector does not reflect the line's length, regardless of whether a vector was used to define the line's length.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
-1.0 >= *Value* >= 1.0	Specifies the Z-component of the line's direction unit vector, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_CENTER_X +	Inquires the X-coordinate of the line's center point. This inquire type is only available if the line's length is not M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-coordinate of the center point on the line, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_CENTER_Y +	Inquires the Y-coordinate of the line's center point. This inquire type is only available if the line's length is not M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-coordinate of the center point on the line, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_CENTER_Z +	Inquires the Z-coordinate of the line's center point. This inquire type is only available if the line's length is not M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Z-coordinate of the center point on the line, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b

M_END_POINT_X +	Inquires the X-coordinate of the line's end point. This inquire type is only available if the line's length is not M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-coordinate of the line's end point, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_END_POINT_Y +	Inquires the Y-coordinate of the line's end point. This inquire type is only available if the line's length is not M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-coordinate of the line's end point, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_END_POINT_Z +	Inquires the Z-coordinate of the line's end point. This inquire type is only available if the line's length is not M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Z-coordinate of the line's end point, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_LENGTH +	Inquires the line's length.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies that the line is infinite.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the line's length in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_START_POINT_X +	Inquires the X-coordinate of the line's start point. When the line's length is infinite, this returns the X-coordinate of the first point used to define the line. SET	c		e	f	g	a		i		j	k	l	m	n	b

		UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the X-coordinate of the first point used to define the line, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_START_POINT_Y +		Inquires the Y-coordinate of the line's start point. When the line's length is infinite, this returns the Y-coordinate of the first point used to define the line. SET	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the Y-coordinate of the first point used to define the line, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_START_POINT_Z +		Inquires the Z-coordinate of the line's start point. When the line's length is infinite, this returns the Z-coordinate of the first point used to define the line. SET	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the Z-coordinate of the first point used to define the line, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
3D plane geometry object ID			Specifies a 3D plane geometry object, allocated using M3dgeoAlloc() with M_GEOMETRY and successfully defined as a plane.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CLOSEST_TO_ORIGIN_X +		Inquires the X-coordinate of the point on the plane closest to the origin of the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the X-coordinate of the point on the plane closest to the origin of the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b

M_CLOSEST_TO_ORIGIN_Y +	Inquires the Y-coordinate of the point on the plane closest to the origin of the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-coordinate of the point on the plane closest to the origin of the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLOSEST_TO_ORIGIN_Z +	Inquires the Z-coordinate of the point on the plane closest to the origin of the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Z-coordinate of the point on the plane closest to the origin of the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_COEFFICIENT_A +	Inquires the coefficient *A* of the plane equation, Ax + By + Cz + D = 0. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
-1.0 >= *Value* >= 1.0	Specifies the coefficient *A* of the plane equation.	c		e	f	g	a		i		j	k	l	m	n	b
M_COEFFICIENT_B +	Inquires the coefficient *B* of the plane equation, Ax + By + Cz + D = 0. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
-1.0 >= *Value* >= 1.0	Specifies the coefficient *B* of the plane equation.	c		e	f	g	a		i		j	k	l	m	n	b
M_COEFFICIENT_C +	Inquires the coefficient *C* of the plane equation, Ax + By + Cz + D = 0. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
-1.0 >= *Value* >= 1.0	Specifies the coefficient *C* of the plane equation.	c		e	f	g	a		i		j	k	l	m	n	b

M_COEFFICIENT_D +	Inquires the coefficient *D* of the plane equation, Ax + By + Cz + D = 0. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the coefficient *D* of the plane equation.	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMAL_X +	Inquires the X-component of the plane's normal unit vector. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
-1.0 >= *Value* >= 1.0	Specifies the X-component of the plane's normal unit vector, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMAL_Y +	Inquires the Y-component of the plane's normal unit vector. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
-1.0 >= *Value* >= 1.0	Specifies the Y-component of the plane's normal unit vector, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMAL_Z +	Inquires the Z-component of the plane's normal unit vector. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
-1.0 >= *Value* >= 1.0	Specifies the Z-component of the plane's normal unit vector, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
3D point geometry object ID	Specifies a 3D point geometry object, allocated using M3dgeoAlloc() with M_GEOMETRY and successfully defined as a point.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_X +	Inquires the X-coordinate of the point. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

		Value	Specifies the X-coordinate of the point, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_POSITION_Y +		Inquires the Y-coordinate of the point. SET	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the Y-coordinate of the point, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_POSITION_Z +		Inquires the Z-coordinate of the point. SET	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the Z-coordinate of the point, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
3D sphere geometry object ID			Specifies a 3D sphere geometry object, allocated using M3dgeoAlloc() with M_GEOMETRY and successfully defined as a sphere.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CENTER_X +		Inquires the X-coordinate of the sphere's center point. SET	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the X-coordinate of the sphere's center point, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CENTER_Y +		Inquires the Y-coordinate of the sphere's center point. SET	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the Y-coordinate of the sphere's center point, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b

M_CENTER_Z +	Inquires the Z-coordinate of the sphere's center point. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Z-coordinate of the sphere's center point, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_RADIUS +	Inquires the sphere's radius. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the sphere's radius in world units.	c		e	f	g	a		i		j	k	l	m	n	b

For any transformation matrix object, the GeometryOrMatrix3dgeoId and InquireType parameters can be set to one of the following:

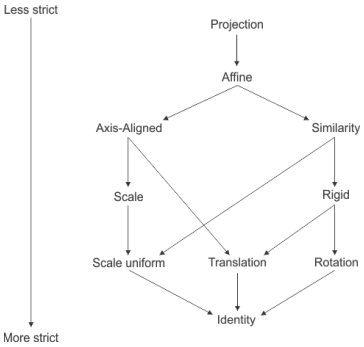
Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about any transformation matrix object																
GeometryOrMatrix3dgeoId	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
InquireType																
UserVarPtr - Possible values returned																
Transformation matrix object ID	Specifies a transformation matrix object, allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.															
M_AFFINE +	Inquires whether the transformation matrix object is an affine transformation matrix. An affine transformation matrix has translation and/or rotation and/or scale transformations. The scale can be uniform or non-uniform.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the transformation matrix object is not an affine transformation matrix.	c		e	f	g	a		i		j	k	l	m	n	b

		M_TRUE	Specifies that the transformation matrix object is an affine transformation matrix.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AXIS_ALIGNED +		Inquires whether the transformation matrix object is an axis-aligned transformation matrix. An axis-aligned transformation matrix has translation and/or scale transformations.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
		M_FALSE	Specifies that the transformation matrix object is not an axis-aligned transformation matrix.	c		e	f	g	a		i		j	k	l	m	n	b
		M_TRUE	Specifies that the transformation matrix object is an axis-aligned transformation matrix.	c		e	f	g	a		i		j	k	l	m	n	b
	M_IDENTITY +		Inquires whether the transformation matrix object is an identity transformation matrix.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
		M_FALSE	Specifies that the transformation matrix object is not an identity matrix.	c		e	f	g	a		i		j	k	l	m	n	b
		M_TRUE	Specifies that the transformation matrix object is an identity matrix.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIRROR +		Inquires whether the transformation matrix object preserves the handedness of the coordinate system. If true, applying the matrix will flip the handedness of the coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
		M_FALSE	Specifies that applying the transformation matrix object will flip the handedness of the coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b

		M_TRUE	Specifies that applying the transformation matrix object preserves the handedness of the coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_RIGID +		Inquires whether the transformation matrix object is a rigid transformation matrix. A rigid transformation matrix has translation and/or rotation transformations.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
		M_FALSE	Specifies that the transformation matrix object is not a rigid transformation matrix.	c		e	f	g	a		i		j	k	l	m	n	b
		M_TRUE	Specifies that the transformation matrix object is a rigid transformation matrix.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ROTATION +		Inquires whether the transformation matrix object is a rotation transformation matrix.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
		M_FALSE	Specifies that the transformation matrix object is not a rotation transformation matrix.	c		e	f	g	a		i		j	k	l	m	n	b
		M_TRUE	Specifies that the transformation matrix object is a rotation transformation matrix.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SCALE +		Inquires whether the transformation matrix object is a scale transformation matrix. The scale can be uniform or non-uniform.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
		M_FALSE	Specifies that the transformation matrix object is not a scale transformation matrix.	c		e	f	g	a		i		j	k	l	m	n	b

		M_TRUE	Specifies that the transformation matrix object is a scale transformation matrix.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SCALE_UNIFORM +		Inquires whether the transformation matrix object is a uniform scale transformation matrix.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
		M_FALSE	Specifies that the transformation matrix object is not a uniform scale transformation matrix.	c		e	f	g	a		i		j	k	l	m	n	b
		M_TRUE	Specifies that the transformation matrix object is a uniform scale transformation matrix.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SIMILARITY +		Inquires whether the transformation matrix object is a similarity transformation matrix. A similarity transformation matrix has translation and/or rotation and/or uniform scale transformations.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
		M_FALSE	Specifies that the transformation matrix object is not a similarity transformation matrix.	c		e	f	g	a		i		j	k	l	m	n	b
		M_TRUE	Specifies that the transformation matrix object is a similarity transformation matrix.	c		e	f	g	a		i		j	k	l	m	n	b

	M_TRANSFORMATION_TYPE +	Inquires the transformation matrix object's transformation type. Note that this inquire type will return the strictest type that matches the transformation matrix object. For example, if the transformation matrix object is a rotation transformation matrix, it will return true for M_ROTATION, M_RIGID, M_SIMILARITY, and M_AFFINE, but M_TRANSFORMATION_TYPE will return M_ROTATION.  	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
		M_AFFINE	Specifies an affine transformation matrix. This is returned if the transformation matrix object has a non-uniform scale transformation, and at least one other type of transformation, including translation or rotation.	c		e	f	g	a		i		j	k	l	m	n	b
		M_AXIS_ALIGNED	Specifies an axis-aligned transformation matrix. This is returned if the transformation matrix object has translation and scale transformations.	c		e	f	g	a		i		j	k	l	m	n	b
		M_IDENTITY	Specifies an identity transformation matrix.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROJECTION	Specifies a projection transformation matrix.	c		e	f	g	a		i		j	k	l	m	n	b
		M_RIGID	Specifies a rigid transformation matrix. This is returned if the transformation matrix object has rotation and translation transformations.	c		e	f	g	a		i		j	k	l	m	n	b

		M_ROTATION	Specifies a rotation transformation matrix.	c		e	f	g	a		i		j	k	l	m	n	b
		M_SCALE	Specifies a scale transformation matrix. This is returned if the transformation matrix object has a non-uniform transformation.	c		e	f	g	a		i		j	k	l	m	n	b
		M_SCALE_UNIFORM	Specifies a uniform scale transformation matrix.	c		e	f	g	a		i		j	k	l	m	n	b
		M_SIMILARITY	Specifies a similarity transformation matrix. This is returned if the transformation matrix object has a uniform scale transformation, and at least one other type of transformation, including translation or rotation.	c		e	f	g	a		i		j	k	l	m	n	b
		M_TRANSLATION	Specifies a translation transformation matrix.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRANSLATION +		Inquires whether the transformation matrix object is a translation transformation matrix.	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
		M_FALSE	Specifies that the transformation matrix object is not a translation transformation matrix.	c		e	f	g	a		i		j	k	l	m	n	b
		M_TRUE	Specifies that the transformation matrix object is a translation transformation matrix.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for
Tables:

- For inquiring about any 3D geometry object
- For inquiring about a specific type of 3D geometry object
- For inquiring about any transformation matrix object

the values listed in all the parameter association tables.

You can add the following value to the above-mentioned values to determine whether an inquire type is supported.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring whether an inquire type is supported

InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_SUPPORTED	Inquires whether the specified inquire type is supported.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the inquire type is not supported.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the inquire type is supported.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for

Tables:

- For inquiring about any 3D geometry object
- For inquiring about a specific type of 3D geometry object
- For inquiring about any transformation matrix object

the values listed in all the parameter association tables.

You can add one of the following values to the above-mentioned values to cast the requested information to the required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested information to an *AIL_DOUBLE*.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_FLOAT	Casts the requested information to an *AIL_FLOAT*.	c		e	f	g	a		i		j	k	l	m	n	b

UserVarPtr extra info	Data type info: Data type: *address of an AIL_FLOAT* **|** *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested information to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT* **|** *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested information to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT32* **|** *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested information to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64* **|** *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]*	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the requested information, cast to an *AIL_DOUBLE*. If the requested information does not fit into an *AIL_DOUBLE*, this function will return **M_NULL** or truncate the information.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3d11.lib.
DLL	Requires ail11.dll; ail3d11.dll.

M3dgeoLine

Synopsis

Define an arbitrary 3D line geometry object.

Syntax

```
void M3dgeoLine(
    AIL_ID Geometry3dgeoId,   //out
    AIL_INT64 CreationMode,   //in
    AIL_DOUBLE XPos1,         //in
    AIL_DOUBLE YPos1,         //in
    AIL_DOUBLE ZPos1,         //in
    AIL_DOUBLE XPos2OrVector, //in
    AIL_DOUBLE YPos2OrVector, //in
    AIL_DOUBLE ZPos2OrVector, //in
    AIL_DOUBLE Length,        //in
    AIL_INT64 ControlFlag     //in
)
```

Description

This function defines an arbitrary 3D line geometry object. You can translate, rotate, scale, or transform the resulting line, using the 3D image processing module.

You can use the resulting line to, for example, perform an arithmetic operation on a depth map using the 3D image processing module, or calculate its distance from each point in a point cloud using the 3D metrology module.

If you want to define a line geometry object from results obtained in a different module, you can use the copy function of that module.

All coordinates are expressed in world units in the working coordinate system.

> Note that if Geometry3dgeoId specifies a previously-defined line, you can leave some of its attributes unchanged, even if that attribute was set using a different creation mode or was modified using the 3D image processing module. To do so, set the corresponding parameter to M_UNCHANGED.

Parameters

Geometry3dgeoId

Specifies the identifier of the 3D geometry object, previously allocated on the required system using M3dgeoAlloc() with M_GEOMETRY.

CreationMode

Specifies how the line is defined.

See the Parameter associations section for possible values that can be specified.

XPos1

Specifies the X-coordinate of the first point used to define the line.

See the Parameter associations section for possible values that can be specified.

YPos1

Specifies the Y-coordinate of the first point used to define the line.

See the Parameter associations section for possible values that can be specified.

ZPos1

Specifies the Z-coordinate of the first point used to define the line.

See the Parameter associations section for possible values that can be specified.

XPos2OrVector

Specifies the X-coordinate of the second point used to define the line, or the X-component of the vector defining the line's direction and length (if Length is M_DEFAULT).

See the Parameter associations section for possible values that can be specified.

YPos2OrVector

Specifies the Y-coordinate of the second point used to define the line, or the Y-component of the vector defining the line's direction and length (if Length is M_DEFAULT).

See the Parameter associations section for possible values that can be specified.

ZPos2OrVector

Specifies the Z-coordinate of the second point used to define the line, or the Z-component of the vector defining the line's direction and length (if Length is M_DEFAULT).

See the Parameter associations section for possible values that can be specified.

Length

Specifies to override the default line length.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying to override the default line length																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default length defined by the creation mode.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies an infinite line.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies to override the line's length with a specific value.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Parameter associations

The table below lists possible values for the **CreationMode**, **XPos1**, **YPos1**, **ZPos1**, **XPos2OrVector**, **YPos2OrVector**, and **ZPos2OrVector** parameters.

- For specifying the line

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the line INQ																	
CreationMode	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
XPos1																
YPos1																
ZPos1																
XPos2OrVector																
YPos2OrVector																
ZPos2OrVector																
M_POINT_AND_VECTOR	Defines the line using a point on the line and a nonzero vector defining the line's direction. By default, the length of the line is finite and is set to the vector's magnitude. The point is at the start of the line. You can determine the coordinates of the line's end point by adding the given vector's X-, Y-, and Z-components to the first point's respective coordinates. You can also use M3dgeoInquire() with M_END_POINT_.... Note that if you set one of the vector's components to M_UNCHANGED, you must also set the other two components to M_UNCHANGED.															
XPos1	Specifies the X-coordinate of the point on the line.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-coordinate of the first point on the line.	c		e	f	g	a		i		j	k	l	m	n	b
YPos1	Specifies the Y-coordinate of the point on the line.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b

Value	Specifies the Y-coordinate of the first point on the line.	c		e	f	g	a		i		j	k	l	m	n	b
ZPos1	Specifies the Z-coordinate of the point on the line.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Z-coordinate of the first point on the line.	c		e	f	g	a		i		j	k	l	m	n	b
XPos2OrVector	Specifies the X-component of the vector defining the line's direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-component of the vector defining the line's direction.	c		e	f	g	a		i		j	k	l	m	n	b
YPos2OrVector	Specifies the Y-component of the vector defining the line's direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-component of the vector defining the line's direction.	c		e	f	g	a		i		j	k	l	m	n	b
ZPos2OrVector	Specifies the Z-component of the vector defining the line's direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Z-component of the vector defining the line's direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_TWO_POINTS	Defines the line using any two non-identical points. By default, the length of the line is finite and is set to the distance between the two specified points. The line starts at the first point, and ends at the second point.	c		e	f	g	a		i		j	k	l	m	n	b
XPos1	Specifies the X-coordinate of the first point on the line.	c		e	f	g	a		i		j	k	l	m	n	b

		M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the X-coordinate of the first point on the line.	c		e	f	g	a		i		j	k	l	m	n	b
	YPos1		Specifies the Y-coordinate of the first point on the line.	c		e	f	g	a		i		j	k	l	m	n	b
		M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the Y-coordinate of the first point on the line.	c		e	f	g	a		i		j	k	l	m	n	b
	ZPos1		Specifies the Z-coordinate of the first point on the line.	c		e	f	g	a		i		j	k	l	m	n	b
		M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the Z-coordinate of the first point on the line.	c		e	f	g	a		i		j	k	l	m	n	b
	XPos2OrVector		Specifies the X-coordinate of the second point on the line.	c		e	f	g	a		i		j	k	l	m	n	b
	YPos2OrVector		Specifies the Y-coordinate of the second point on the line.	c		e	f	g	a		i		j	k	l	m	n	b
	ZPos2OrVector		Specifies the Z-coordinate of the second point on the line.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3d11.lib.
DLL	Requires ail11.dll; ail3d11.dll.

M3dgeoMatrixGet

Synopsis

Get data from a transformation matrix object and place it in a user-supplied array.

Syntax

```
void M3dgeoMatrixGet(
    AIL_ID Matrix3dgeoId,    //in
    AIL_INT64 GetType,       //in
    AIL_DOUBLE *UserArrayPtr //out
)
```

Description

This function copies the values from a transformation matrix object and places them in an array.

> Note that presented above is an example prototype of this function. You can call M3dgeoMatrixGet() with the same prototype as any of the type-specific functions below, without using the function's type-specific name; that is, use M3dgeoMatrixGet() and not, for example, M3dgeoMatrixGetFloat(). The data is automatically cast into the appropriate type. This is true except when using a C compiler. When using a C compiler, you must always call this function with its type-specific name (you cannot use the standard name of this function).

Parameters

Matrix3dgeoId

Specifies the identifier of the source transformation matrix object. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.

GetType

Reserved for future expansion and must be set to **M_DEFAULT**.

UserArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(16)

Specifies the address of the user array in which to copy the data from the transformation matrix. Ensure that the user array is large enough to receive the data to be copied from the source object. Transformation matrices are 4x4 matrices, so the array must be of size 16.

Type-specific versions of the function when using a C compiler

```
void M3dgeoMatrixGetDouble (AIL_ID Matrix3dgeoId, AIL_INT64 GetType, AIL_DOUBLE *UserArrayPtr)
```

Details

Parameters

Matrix3dgeoId

See Matrix3dgeoId of the main function for a description.

GetType

See GetType of the main function for a description.

UserArrayPtr

See UserArrayPtr of the main function for a description.

```
void M3dgeoMatrixGetFloat (AIL_ID Matrix3dgeoId, AIL_INT64 GetType, AIL_FLOAT *UserArrayPtr)
```

Details

Parameters

Matrix3dgeoId

See Matrix3dgeoId of the main function for a description.

GetType

See GetType of the main function for a description.

UserArrayPtr

See UserArrayPtr of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3d11.lib.
DLL	Requires ail11.dll; ail3d11.dll.

M3dgeoMatrixGetTransform

Synopsis

Get the translation, rotation, or scale defined by a given transformation matrix object.

Syntax

```
void M3dgeoMatrixGetTransform(
    AIL_ID Matrix3dgeoId,  //in
    AIL_INT64 InquireType, //in
    AIL_DOUBLE *Param1Ptr, //out
    AIL_DOUBLE *Param2Ptr, //out
    AIL_DOUBLE *Param3Ptr, //out
    AIL_DOUBLE *Param4Ptr, //out
    AIL_INT64 ControlFlag  //in
)
```

Description

This function retrieves the translation, rotation, or scale defined by a given transformation matrix object.

The three distinct rotations about the axes of the working coordinate system (M_ROTATION_...) are also known as roll, pitch, and yaw. You can determine the required rotation order by checking the convention used by your robot's manufacturer. Note that M_ROTATION_AXIS... and M_ROTATION_QUATERNION do not depend on the manufacturer.

Note that **M3dgeoMatrixGetTransform**() returns angles in degrees. However, unlike most other Aurora Imaging Library functions, this function returns angles following the right-hand grip rule around the axis of rotation; if you wrap your right hand around the axis of rotation, pointing your thumb in the positive direction of the axis, your fingers wrap in the direction of rotation. For example, a positive rotation around the Z-axis corresponds to a rotation that turns the positive X-axis toward the positive Y-axis.

All coordinates are expressed in world units in the working coordinate system.

Parameters

Matrix3dgeoId

Specifies the identifier of the transformation matrix object. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.

> This function only supports matrices that can qualify as matrices of any type other than M_AFFINE or M_PROJECTION. A transformation matrix object is supported if passing it in a call to M3dgeoInquire() with the M_TRANSFORMATION_TYPE inquire type returns any value other than M_AFFINE or M_PROJECTION.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the transformation matrix object identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

Transformation matrix object identifier	Specifies the identifier of a 3D geometry object.	c		e	f	g	a		i		j	k	l	m	n	b

InquireType

Specifies the type of transformation to retrieve from the given transformation matrix object.

See the Parameter associations section for possible values that can be specified.

Param1Ptr

Specifies the first address in which to return information about the transformation required.

See the Parameter associations section for possible values that can be returned.

Param2Ptr

Specifies the second address in which to return information about the transformation required.

See the Parameter associations section for possible values that can be returned.

Param3Ptr

Specifies the third address in which to return information about the transformation required.

See the Parameter associations section for possible values that can be returned.

Param4Ptr

Specifies the fourth address in which to return information about the transformation required.

If this parameter is not used, it must be set to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Parameter associations

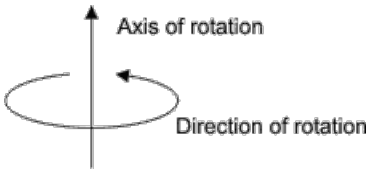
The table below lists possible values for the **InquireType** parameter and possible values returned to the **Param1Ptr**, **Param2Ptr**, **Param3Ptr**, and **Param4Ptr** parameters.

- For specifying the transformation type

Any parameters for which a value is not returned must be set to **M_NULL**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the transformation type																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	Param1Ptr - Possible values returned																

	Param2Ptr - Possible values returned															
	Param3Ptr - Possible values returned															
	Param4Ptr - Possible values returned															
M_ROTATION_AXIS_ANGLE		Retrieves the axis and angle of rotation. The axis of rotation is defined by a vector. The rotation angle is measured in the counter-clockwise direction around the axis of rotation, as per the right-hand rule.  The axis of rotation is always normalized. SET														
	Param1Ptr	Returns the X-component of the vector that defines the axis of rotation.	c		e	f	g	a		i		j	k	l	m	n	b
	Param2Ptr	Returns the Y-component of the vector that defines the axis of rotation.	c		e	f	g	a		i		j	k	l	m	n	b
	Param3Ptr	Returns the Z-component of the vector that defines the axis of rotation.	c		e	f	g	a		i		j	k	l	m	n	b
	Param4Ptr	Returns the angle of the rotation, in degrees, around the axis of rotation.	c		e	f	g	a		i		j	k	l	m	n	b
		0 <= *Value* < 360	Specifies the rotation angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_AXIS_X		Retrieves the normalized vector of the X-axis which results from applying the given transformation matrix to the working coordinate system's X-axis. SET	c		e	f	g	a		i		j	k	l	m	n	b
	Param1Ptr	Returns the X-component of the vector.	c		e	f	g	a		i		j	k	l	m	n	b
	Param2Ptr	Returns the Y-component of the vector.	c		e	f	g	a		i		j	k	l	m	n	b
	Param3Ptr	Returns the Z-component of the vector.	c		e	f	g	a		i		j	k	l	m	n	b

M_ROTATION_AXIS_Y	Retrieves the normalized vector of the Y-axis which results from applying the given transformation matrix to the working coordinate system's Y-axis. SET	c		e	f	g	a		i		j	k	l	m	n	b
Param1Ptr	Returns the X-component of the vector.	c		e	f	g	a		i		j	k	l	m	n	b
Param2Ptr	Returns the Y-component of the vector.	c		e	f	g	a		i		j	k	l	m	n	b
Param3Ptr	Returns the Z-component of the vector.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_AXIS_Z	Retrieves the normalized vector of the Z-axis which results from applying the given transformation matrix to the working coordinate system's Z-axis. SET	c		e	f	g	a		i		j	k	l	m	n	b
Param1Ptr	Returns the X-component of the vector.	c		e	f	g	a		i		j	k	l	m	n	b
Param2Ptr	Returns the Y-component of the vector.	c		e	f	g	a		i		j	k	l	m	n	b
Param3Ptr	Returns the Z-component of the vector.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_QUATERNION	Retrieves the components of the rotation quaternion. The quaternion is always normalized. SET	c		e	f	g	a		i		j	k	l	m	n	b
Param1Ptr	Returns the scalar component of the quaternion.	c		e	f	g	a		i		j	k	l	m	n	b
Param2Ptr	Returns the X-component of the quaternion.	c		e	f	g	a		i		j	k	l	m	n	b
Param3Ptr	Returns the Y-component of the quaternion.	c		e	f	g	a		i		j	k	l	m	n	b
Param4Ptr	Returns the Z-component of the quaternion.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_XYZ	Retrieves the three distinct rotations about the axes of the working coordinate system in the following order: the X-axis, the Y-axis, and the Z-axis rotation. SET	c		e	f	g	a		i		j	k	l	m	n	b
Param1Ptr	Returns the X-axis rotation.	c		e	f	g	a		i		j	k	l	m	n	b

	0.0 <= *Value* < 360.0	Specifies the X-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
Param2Ptr		Returns the Y-axis rotation.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 90.0	Specifies the Y-axis rotation, in degrees, when located between the positive Z-axis and the positive X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
	270.0 <= *Value* < 360.0	Specifies the Y-axis rotation, in degrees, when located between the positive Z-axis and the negative X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
Param3Ptr		Returns the Z-axis rotation.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* < 360.0	Specifies the Z-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_XZY		Retrieves the three distinct rotations about the axes of the working coordinate system in the following order: the X-axis, the Z-axis, and the Y-axis rotation. SET	c		e	f	g	a		i		j	k	l	m	n	b
Param1Ptr		Returns the X-axis rotation.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* < 360.0	Specifies the X-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
Param2Ptr		Returns the Z-axis rotation.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 90.0	Specifies the Z-axis rotation, in degrees, when located between the positive X-axis and the positive Y-axis.	c		e	f	g	a		i		j	k	l	m	n	b
	270.0 <= *Value* < 360.0	Specifies the Z-axis rotation, in degrees, when located between the positive X-axis and the negative Y-axis.	c		e	f	g	a		i		j	k	l	m	n	b
Param3Ptr		Returns the Y-axis rotation.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* < 360.0	Specifies the Y-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_YXZ		Retrieves the three distinct rotations about the axes of the working coordinate system in the following order: the Y-axis, the X-axis, and the Z-axis rotation. SET	c		e	f	g	a		i		j	k	l	m	n	b

Param1Ptr	Returns the Y-axis rotation.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* < 360.0	Specifies the Y-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
Param2Ptr	Returns the X-axis rotation.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 90.0	Specifies the X-axis rotation, in degrees, when located between the positive Y-axis and the positive Z-axis.	c		e	f	g	a		i		j	k	l	m	n	b
270.0 <= *Value* < 360.0	Specifies the X-axis rotation, in degrees, when located between the positive Y-axis and the negative Z-axis.	c		e	f	g	a		i		j	k	l	m	n	b
Param3Ptr	Returns the Z-axis rotation.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* < 360.0	Specifies the Z-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_YZX	Retrieves the three distinct rotations about the axes of the working coordinate system in the following order: the Y-axis, the Z-axis, and the X-axis rotation. SET	c		e	f	g	a		i		j	k	l	m	n	b
Param1Ptr	Returns the Y-axis rotation.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* < 360.0	Specifies the Y-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
Param2Ptr	Returns the Z-axis rotation.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 90.0	Specifies the Z-axis rotation, in degrees, when located between the positive X-axis and the positive Y-axis.	c		e	f	g	a		i		j	k	l	m	n	b
270.0 <= *Value* < 360.0	Specifies the Z-axis rotation, in degrees, when located between the positive X-axis and the negative Y-axis.	c		e	f	g	a		i		j	k	l	m	n	b
Param3Ptr	Returns the X-axis rotation.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* < 360.0	Specifies the X-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b

M_ROTATION_ZXY	Retrieves the three distinct rotations about the axes of the working coordinate system in the following order: the Z-axis, the X-axis, and the Y-axis rotation. SET	c		e	f	g	a		i		j	k	l	m	n	b
Param1Ptr	Returns the Z-axis rotation.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* < 360.0	Specifies the Z-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
Param2Ptr	Returns the X-axis rotation.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 90.0	Specifies the X-axis rotation, in degrees, when located between the positive Y-axis and the positive Z-axis.	c		e	f	g	a		i		j	k	l	m	n	b
270.0 <= *Value* < 360.0	Specifies the X-axis rotation, in degrees, when located between the positive Y-axis and the negative Z-axis.	c		e	f	g	a		i		j	k	l	m	n	b
Param3Ptr	Returns the Y-axis rotation.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* < 360.0	Specifies the Y-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_ZYX	Retrieves the three distinct rotations about the axes of the working coordinate system in the following order: the Z-axis, the Y-axis, and the X-axis rotation. SET	c		e	f	g	a		i		j	k	l	m	n	b
Param1Ptr	Returns the Z-axis rotation.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* < 360.0	Specifies the Z-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
Param2Ptr	Returns the Y-axis rotation.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 90.0	Specifies the Y-axis rotation, in degrees, when located between the positive Z-axis and the positive X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
270.0 <= *Value* < 360.0	Specifies the Y-axis rotation, in degrees, when located between the positive Z-axis and the negative X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
Param3Ptr	Returns the X-axis rotation.	c		e	f	g	a		i		j	k	l	m	n	b

	0.0 <= *Value* < 360.0	Specifies the X-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCALE		Retrieves the scale factor along each axis. SET	c		e	f	g	a		i		j	k	l	m	n	b
	Param1Ptr	Returns the scaling factor in X.	c		e	f	g	a		i		j	k	l	m	n	b
	Param2Ptr	Returns the scaling factor in Y.	c		e	f	g	a		i		j	k	l	m	n	b
	Param3Ptr	Returns the scaling factor in Z.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRANSLATION		Retrieves the translation along each of the axes of the working coordinate system. SET	c		e	f	g	a		i		j	k	l	m	n	b
	Param1Ptr	Returns the displacement along the X-axis of the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	Param2Ptr	Returns the displacement along the Y-axis of the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	Param3Ptr	Returns the displacement along the Z-axis of the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3d11.lib.
DLL	Requires ail11.dll; ail3d11.dll.

M3dgeoMatrixPut

Synopsis

Put data from a user-supplied array into a transformation matrix object.

Syntax

```
void M3dgeoMatrixPut(
    AIL_ID Matrix3dgeoId,          //out
    AIL_INT64 PutType,             //in
    const AIL_DOUBLE *UserArrayPtr //in
)
```

Description

This function copies an array of values into a specified transformation matrix object.

> Note that presented above is an example prototype of this function. You can call M3dgeoMatrixPut() with the same prototype as any of the type-specific functions below, without using the function's type-specific name; that is, use M3dgeoMatrixPut() and not, for example, M3dgeoMatrixPutFloat(). The data is automatically cast into the appropriate type. This is true except when using a C compiler. When using a C compiler, you must always call this function with its type-specific name (you cannot use the standard name of this function).

Parameters

Matrix3dgeoId

Specifies the identifier of the transformation matrix object. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.

PutType

Reserved for future expansion and must be set to **M_DEFAULT**.

UserArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(16)

Specifies the address of the user array from which to copy the data into the transformation matrix object. Ensure that user array is large enough to contain the data to be copied to the destination object. Transformation matrices are 4x4 matrices, so the array must be of size 16.

Type-specific versions of the function when using a C compiler

```
void M3dgeoMatrixPutDouble (AIL_ID Matrix3dgeoId, AIL_INT64 PutType, const AIL_DOUBLE *UserArrayPtr)
```

Details

Parameters

Matrix3dgeoId

See Matrix3dgeoId of the main function for a description.

PutType

See PutType of the main function for a description.

UserArrayPtr

See UserArrayPtr of the main function for a description.

```
void M3dgeoMatrixPutFloat (AIL_ID Matrix3dgeoId, AIL_INT64 PutType, const AIL_FLOAT *UserArrayPtr)
```

Details

Parameters

Matrix3dgeoId

See Matrix3dgeoId of the main function for a description.

PutType

See PutType of the main function for a description.

UserArrayPtr

See UserArrayPtr of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3d11.lib.
DLL	Requires ail11.dll; ail3d11.dll.

M3dgeoMatrixSetTransform

Synopsis

Set the transformation values of a transformation matrix object.

Syntax

```
void M3dgeoMatrixSetTransform(
    AIL_ID Matrix3dgeoId,    //out
    AIL_INT64 TransformType, //in
    AIL_DOUBLE Param1,       //in
    AIL_DOUBLE Param2,       //in
    AIL_DOUBLE Param3,       //in
    AIL_DOUBLE Param4,       //in
    AIL_INT64 ControlFlag    //in
)
```

Description

This function assigns a transformation to a transformation matrix object or composes a transformation with a transformation matrix object. The transformation can be defined as a translation, rotation, scale, or the result of a matrix composition.

The three distinct rotations about the axes of the working coordinate system (M_ROTATION_...) are also known as roll, pitch, and yaw. You can determine the required rotation order by checking the convention used by your robot's manufacturer. Note that M_ROTATION_AXIS... and M_ROTATION_QUATERNION do not depend on the manufacturer.

Note that all angles should be given in degrees. However, unlike most other Aurora Imaging Library functions, angles are interpreted using the right-hand grip rule around the axis of rotation; if you wrap your right hand around the axis of rotation, pointing your thumb in the positive direction of the axis, your fingers wrap in the direction of rotation. For example, a positive rotation around the Z-axis corresponds to a rotation that turns the positive X-axis toward the positive Y-axis.

All coordinates are expressed in world units in the working coordinate system.

Parameters

Matrix3dgeoId

Specifies the identifier of the transformation matrix object. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.

TransformType

Specifies the type of transformation to apply.

See the Parameter associations section for possible values that can be specified.

Param1

Specifies the first parameter used to define the transformation matrix.

See the Parameter associations section for possible values that can be specified.

Param2

Specifies the second parameter used to define the transformation matrix.

Set this parameter to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

Param3

Specifies the third parameter used to define the transformation matrix.

Set this parameter to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

Param4

Specifies the fourth parameter used to define the transformation matrix.

Set this parameter to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Specifies how to apply the transformation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying how to apply the transformation																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_ASSIGN.	c		e	f	g	a		i		j	k	l	m	n	b
M_ASSIGN	Specifies to assign the specified transformation to the given transformation matrix object.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPOSE_WITH_CURRENT	Specifies to compose the specified transformation with the given matrix. The matrix composition follows the equation BA = C, where *A* is the transformation matrix specified by Matrix3dgeoId, *B* is the transformation matrix defined by the parameter associations, and *C* is the resulting transformation matrix. The values of the resulting transformation matrix are written to the transformation matrix object specified by Matrix3dgeoId.	c		e	f	g	a		i		j	k	l	m	n	b

Parameter associations

The table below lists possible values for the **TransformType**, **Param1**, **Param2**, **Param3**, and **Param4**

parameters.

- For specifying the transformation type

Set unused parameters to **M_DEFAULT**.

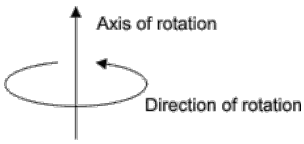
Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the transformation type																	
TransformType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Param1																
Param2																
Param3																
Param4																
M_COMPOSE_TWO_MATRICES	Specifies the transformation resulting from the composition of two matrices. The matrix composition follows the equation AB = C, where *A* is the transformation matrix specified by Param1, *B* is the transformation matrix specified by Param2, and *C* is the resulting transformation matrix. Note that transformation matrix *B* is applied before transformation matrix *A*. Therefore, transformation matrix *C* describes the transformation obtained by applying transformation matrix *B* followed by transformation matrix *A*.															
Param1	Specifies the first source matrix (that is, source matrix *A*).	c		e	f	g	a		i		j	k	l	m	n	b
M_IDENTITY_MATRIX	Specifies the identity matrix.	c		e	f	g	a		i		j	k	l	m	n	b
Transformation matrix object identifier	Specifies the identifier of a transformation matrix object.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Specifies the second source matrix (that is, source matrix *B*).	c		e	f	g	a		i		j	k	l	m	n	b
M_IDENTITY_MATRIX	Specifies the identity matrix.	c		e	f	g	a		i		j	k	l	m	n	b

		Transformation matrix object identifier	Specifies the identifier of a transformation matrix object.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIXTURE_TO_GEOMETRY			Specifies a transformation that can move the working coordinate system to a 3D geometry object.	c		e	f	g	a		i		j	k	l	m	n	b
	Param1		Specifies a 3D geometry object.	c		e	f	g	a		i		j	k	l	m	n	b
		3D box geometry object identifier	Specifies a 3D box geometry object identifier previously allocated using M3dgeoAlloc() with M_GEOMETRY and defined as a box. The result is the transformation that can move the working coordinate system such that its origin moves to the box's center, and the box's orientation is applied.	c		e	f	g	a		i		j	k	l	m	n	b
		3D cylinder geometry object identifier	Specifies a 3D cylinder geometry object identifier previously allocated using M3dgeoAlloc() with M_GEOMETRY and defined as a cylinder. The result is the transformation that can move the working coordinate system such that its origin moves to the cylinder's start point, and the cylinder's central axis becomes the positive Z-axis. The rotation of the X- and Y-axis is determined by the minimal rotation that must be applied to align the positive Z-axis with the cylinder's central axis.	c		e	f	g	a		i		j	k	l	m	n	b

		3D plane geometry object identifier	Specifies a 3D plane geometry object identifier previously allocated using M3dgeoAlloc() with M_GEOMETRY and defined as a plane. The result is the transformation that can move the working coordinate system such that its XY (Z=0) plane is moved to the specified plane, and the plane's normal becomes the positive Z-axis. The working coordinate system's origin is moved to the closest point on the plane (the projection).	c		e	f	g	a		i		j	k	l	m	n	b
		3D point geometry object identifier	Specifies a 3D point geometry object identifier previously allocated using M3dgeoAlloc() with M_GEOMETRY and defined as a point. The result is the transformation that can move the working coordinate system such that its origin is moved to the point.	c		e	f	g	a		i		j	k	l	m	n	b
		3D sphere geometry object identifier	Specifies a 3D sphere geometry object identifier previously allocated using M3dgeoAlloc() with M_GEOMETRY and defined as a sphere. The result is the transformation that can move the working coordinate system such that its origin is moved to the sphere's center. No rotation is applied.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIXTURE_TO_PLANE			Specifies a transformation that can move the XY (Z=0) plane of the working coordinate system to a plane specified by Param1. By default, the origin is moved using the same transformation. You can optionally specify a point on the plane where the origin should be moved, using Param2, Param3, and Param4. If the point is not on the plane, or if you set Param2, Param3, and Param4 to **M_DEFAULT**, the closest point on the plane (the projection) is chosen instead.	c		e	f	g	a		i		j	k	l	m	n	b

Param1	Specifies a 3D geometry plane.	c		e	f	g	a		i		j	k	l	m	n	b
M_XY_PLANE	Specifies the XY (Z=0) plane. The result is the transformation that can move the origin of the working coordinate system to a point on its XY (Z=0) plane, specified by Param2, Param3, and Param4. If these parameters are set to **M_DEFAULT**, the result is the identity transformation.	c		e	f	g	a		i		j	k	l	m	n	b
3D geometry object identifier	Specifies a 3D plane geometry object identifier previously allocated using M3dgeoAlloc() with M_GEOMETRY and defined as a plane.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Specifies the X-coordinate of a point on the plane where the origin will be moved.	c		e	f	g	a		i		j	k	l	m	n	b
Param3	Specifies the Y-coordinate of a point on the plane where the origin will be moved.	c		e	f	g	a		i		j	k	l	m	n	b
Param4	Specifies the Z-coordinate of a point on the plane where the origin will be moved.	c		e	f	g	a		i		j	k	l	m	n	b
M_IDENTITY	Specifies the identity transformation. This resets the matrix to the identity matrix, removing all transformations from the transformation matrix object, when ControlFlag is M_ASSIGN.	c		e	f	g	a		i		j	k	l	m	n	b
M_INVERSE	Specifies the inverse transformation.	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Specifies the source matrix.	c		e	f	g	a		i		j	k	l	m	n	b
Transformation matrix object identifier	Specifies the identifier of a transformation matrix object previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b

M_ROTATION_AXIS_ANGLE	Specifies a rotation described by an axis and angle of rotation. The axis of rotation is defined by a vector. The angle of rotation is measured in the counter-clockwise direction around the axis of rotation. INQ 	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Specifies the X-component of the vector that defines the axis of rotation.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Specifies the Y-component of the vector that defines the axis of rotation.	c		e	f	g	a		i		j	k	l	m	n	b
Param3	Specifies the Z-component of the vector that defines the axis of rotation.	c		e	f	g	a		i		j	k	l	m	n	b
Param4	Specifies the angle of the rotation, in degrees, around the axis of rotation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_AXIS_X	Specifies the minimal rotation that can align the working coordinate system's X-axis with the specified vector. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Specifies the X-component of the vector.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Specifies the Y-component of the vector.	c		e	f	g	a		i		j	k	l	m	n	b
Param3	Specifies the Z-component of the vector.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_AXIS_Y	Specifies the minimal rotation that can align the working coordinate system's Y-axis with the specified vector. INQ	c		e	f	g	a		i		j	k	l	m	n	b

Param1	Specifies the X-component of the vector.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Specifies the Y-component of the vector.	c		e	f	g	a		i		j	k	l	m	n	b
Param3	Specifies the Z-component of the vector.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_AXIS_Z	Specifies the minimal rotation that can align the working coordinate system's Z-axis with the specified vector. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Specifies the X-component of the vector.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Specifies the Y-component of the vector.	c		e	f	g	a		i		j	k	l	m	n	b
Param3	Specifies the Z-component of the vector.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_QUATERNION	Specifies a rotation operation that is described by a rotation quaternion. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Specifies the scalar component of the quaternion.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Specifies the X-component of the quaternion.	c		e	f	g	a		i		j	k	l	m	n	b
Param3	Specifies the Y-component of the quaternion.	c		e	f	g	a		i		j	k	l	m	n	b
Param4	Specifies the Z-component of the quaternion.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_X	Specifies a rotation operation around the X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Specifies the X-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b

M_ROTATION_XYZ	Specifies a rotation that is described by three distinct rotations about the axes of the working coordinate system in the following order: a rotation about the X-axis, a rotation about the Y-axis, and a rotation about the Z-axis rotation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Specifies the X-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Specifies the Y-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
Param3	Specifies the Z-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_XZY	Specifies a rotation that is described by three distinct rotations about the axes of the working coordinate system in the following order: a rotation about the X-axis, a rotation about the Z-axis, and a rotation about the Y-axis rotation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Specifies the X-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Specifies the Z-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
Param3	Specifies the Y-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_Y	Specifies a rotation operation around the Y-axis.	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Specifies the Y-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b

M_ROTATION_YXZ	Specifies a rotation that is described by three distinct rotations about the axes of the working coordinate system in the following order: a rotation about the Y-axis, a rotation about the X-axis, and a rotation about the Z-axis rotation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Specifies the Y-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Specifies the X-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
Param3	Specifies the Z-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_YZX	Specifies a rotation that is described by three distinct rotations about the axes of the working coordinate system in the following order: a rotation about the Y-axis, a rotation about the Z-axis, and a rotation about the X-axis rotation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Specifies the Y-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Specifies the Z-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
Param3	Specifies the X-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_Z	Specifies a rotation operation around the Z-axis.	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Specifies the Z-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b

M_ROTATION_ZXY	Specifies a rotation that is described by three distinct rotations about the axes of the working coordinate system in the following order: a rotation about the Z-axis, a rotation about the X-axis, and a rotation about the Y-axis rotation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Specifies the Z-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Specifies the X-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
Param3	Specifies the Y-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_ZYX	Specifies a rotation that is described by three distinct rotations about the axes of the working coordinate system in the following order: a rotation about the Z-axis, a rotation about the Y-axis, and a rotation about the X-axis rotation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Specifies the Z-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Specifies the Y-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
Param3	Specifies the X-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCALE	Specifies a scaling operation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Specifies the scaling factor in X.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Specifies the scaling factor in Y.	c		e	f	g	a		i		j	k	l	m	n	b
Param3	Specifies the scaling factor in Z.	c		e	f	g	a		i		j	k	l	m	n	b

M_TRANSLATION		Specifies a translation along each of the axes of the working coordinate system. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	Param1	Specifies the displacement along the X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
	Param2	Specifies the displacement along the Y-axis.	c		e	f	g	a		i		j	k	l	m	n	b
	Param3	Specifies the displacement along the Z-axis.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3d11.lib.
DLL	Requires ail11.dll; ail3d11.dll.

M3dgeoMatrixSetWithAxes

Synopsis

Defines a transformation matrix that can transform points from the working coordinate system into the specified coordinate system, or vice versa.

Syntax

```
void M3dgeoMatrixSetWithAxes(
    AIL_ID Matrix3dgeoId,    //out
    AIL_INT64 Mode,          //in
    AIL_DOUBLE OriginX,      //in
    AIL_DOUBLE OriginY,      //in
    AIL_DOUBLE OriginZ,      //in
    AIL_DOUBLE Axis1VectorX, //in
    AIL_DOUBLE Axis1VectorY, //in
    AIL_DOUBLE Axis1VectorZ, //in
    AIL_DOUBLE Axis2VectorX, //in
    AIL_DOUBLE Axis2VectorY, //in
    AIL_DOUBLE Axis2VectorZ, //in
    AIL_INT64 ControlFlag    //in
)
```

Description

This function defines a transformation matrix that can transform points from the working coordinate system into a new specified coordinate system, or vice versa.

The new coordinate system is described by its origin and its orientation with respect to the working coordinate system. The Mode parameter specifies which two axes are used to define the new coordinate system. The first axis is defined using Axis1VectorX, Axis1VectorY, and Axis1VectorZ. The second axis is defined using Axis2VectorX, Axis2VectorY, and Axis2VectorZ. The components of the vector aligned with the second axis can be approximate, since this axis will be internally orthogonalized. However, the second axis must not be parallel to the first axis. The third axis is internally deduced by computing the vector that is orthogonal to the first two axes and respects the right-hand rule.

By default, this function returns the transformation matrix that transforms points from the working coordinate system into the new coordinate system. To get the transformation matrix that transforms the points from the new coordinate system into the working coordinate system, use M_COORDINATE_SYSTEM_TRANSFORMATION.

> Note that the two specified axes must have a magnitude that is greater than 0.

Parameters

Matrix3dgeoId

Specifies the identifier of the transformation matrix object, previously allocated on the required system using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.

Mode

Specifies which two axes to use to define the new coordinate system. The first letter determines the first

axis, and the second letter determines the second axis.

> Note that because the second specified axis is internally orthogonalized, the order in which the axes are specified is important. The second specified axis will be moved if it is not already orthogonal to the first specified axis.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

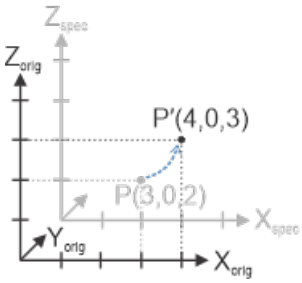
For specifying the two axes to use																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_XY_AXES +	Specifies the X-axis, and then the Y-axis. The X-axis is specified by Axis1VectorX, Axis1VectorY, and Axis1VectorZ. The Y-axis is approximately specified by Axis2VectorX, Axis2VectorY, and Axis2VectorZ.	c		e	f	g	a		i		j	k	l	m	n	b
M_XZ_AXES +	Specifies the X-axis, and then the Z-axis. The X-axis is specified by Axis1VectorX, Axis1VectorY, and Axis1VectorZ. The Z-axis is approximately specified by Axis2VectorX, Axis2VectorY, and Axis2VectorZ.	c		e	f	g	a		i		j	k	l	m	n	b
M_YX_AXES +	Specifies the Y-axis, and then the X-axis. The Y-axis is specified by Axis1VectorX, Axis1VectorY, and Axis1VectorZ. The X-axis is approximately specified by Axis2VectorX, Axis2VectorY, and Axis2VectorZ.	c		e	f	g	a		i		j	k	l	m	n	b
M_YZ_AXES +	Specifies the Y-axis, and then the Z-axis. The Y-axis is specified by Axis1VectorX, Axis1VectorY, and Axis1VectorZ. The Z-axis is approximately specified by Axis2VectorX, Axis2VectorY, and Axis2VectorZ.	c		e	f	g	a		i		j	k	l	m	n	b
M_ZX_AXES +	Specifies the Z-axis, and then the X-axis. The Z-axis is specified by Axis1VectorX, Axis1VectorY, and Axis1VectorZ. The X-axis is approximately specified by Axis2VectorX, Axis2VectorY, and Axis2VectorZ.	c		e	f	g	a		i		j	k	l	m	n	b
M_ZY_AXES +	Specifies the Z-axis, and then the Y-axis. The Z-axis is specified by Axis1VectorX, Axis1VectorY, and Axis1VectorZ. The Y-axis is approximately specified by Axis2VectorX, Axis2VectorY, and Axis2VectorZ.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For specifying the two axes to use.

You can add one of the following values to the above-mentioned values to specify the direction of the transformation between coordinate systems.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the direction of the transformation between coordinate systems																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_DEFAULT	Same as M_POSITION_TRANSFORMATION.	c		e	f	g	a		i		j	k	l	m	n	b
M_COORDINATE_SYSTEM_TRANSFORMATION	Specifies to return the transformation matrix that transforms points such that their coordinates are expressed relative to the specified position and orientation. When you apply the resulting matrix, point coordinates are passively transformed, and will be expressed relative to the newly specified origin and axes. This is a backward mapping that you can use to fixture points to a new coordinate system. In the example below, the point P (3, 0, 2) is expressed as P' (2, 0, 1) in the new coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_TRANSFORMATION	Specifies to return the transformation matrix that transforms points such that they are offset by the specified position and orientation. When you apply the resulting matrix, point coordinates are actively transformed, and will be expressed relative to the original origin and axes. This is a forward mapping that you can use to move points. In the example below, the point P (3, 0, 2) is moved to P' (4, 0, 3). Note that P's offset from the original coordinate system is equivalent to P''s offset from the new coordinate system. 	c		e	f	g	a		i		j	k	l	m	n	b

OriginX

Specifies the X-coordinate of the new coordinate system's origin, expressed in the working coordinate system.

OriginY

Specifies the Y-coordinate of the new coordinate system's origin, expressed in the working coordinate system.

OriginZ

Specifies the Z-coordinate of the new coordinate system's origin, expressed in the working coordinate system.

Axis1VectorX

Specifies the X-component of the vector aligned with the new coordinate system's first axis, expressed in the working coordinate system.

Axis1VectorY

Specifies the Y-component of the vector aligned with the new coordinate system's first axis, expressed in the working coordinate system.

Axis1VectorZ

Specifies the Z-component of the vector aligned with the new coordinate system's first axis, expressed in the working coordinate system.

Axis2VectorX

Specifies the approximate X-component of the vector aligned with the new coordinate system's second axis, expressed in the working coordinate system.

Axis2VectorY

Specifies the approximate Y-component of the vector aligned with the new coordinate system's second axis, expressed in the working coordinate system.

Axis2VectorZ

Specifies the approximate Z-component of the vector aligned with the new coordinate system's second axis, expressed in the working coordinate system.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3d11.lib.
DLL	Requires ail11.dll; ail3d11.dll.

M3dgeoPlane

Synopsis

Define a 3D plane geometry object.

Syntax

```
void M3dgeoPlane(
    AIL_ID Geometry3dgeoId, //out
    AIL_INT64 CreationMode, //in
    AIL_DOUBLE X1,          //in
    AIL_DOUBLE Y1,          //in
    AIL_DOUBLE Z1,          //in
    AIL_DOUBLE X2OrD,       //in
    AIL_DOUBLE Y2,          //in
    AIL_DOUBLE Z2,          //in
    AIL_DOUBLE X3,          //in
    AIL_DOUBLE Y3,          //in
    AIL_DOUBLE Z3,          //in
    AIL_INT64 ControlFlag   //in
)
```

Description

This function defines a 3D plane geometry object. You can translate, rotate, or transform the resulting plane, using the 3D image processing module.

You can use the resulting plane to, for example, crop a point cloud or perform an arithmetic operation on a depth map using the 3D image processing module, or calculate its distance from each point in a point cloud using the 3D metrology module.

If you want to define a plane geometry object from results obtained in a different module, you can use the copy function of that module.

All coordinates are expressed in world units in the working coordinate system.

> Note that if Geometry3dgeoId specifies a previously-defined plane, and CreationMode is set to M_POINT_AND_NORMAL, you can leave the components of the plane's normal vector unchanged, even if the plane was modified using the 3D image processing module.

Parameters

Geometry3dgeoId

Specifies the identifier of the 3D geometry object, previously allocated on the required system using M3dgeoAlloc() with M_GEOMETRY.

CreationMode

Specifies how the plane is defined.

See the Parameter associations section for possible values that can be specified.

X1

Specifies the first parameter used to define the plane.

See the Parameter associations section for possible values that can be specified.

Y1

Specifies the second parameter used to define the plane.

See the Parameter associations section for possible values that can be specified.

Z1

Specifies the third parameter used to define the plane.

See the Parameter associations section for possible values that can be specified.

X2OrD

Specifies the fourth parameter used to define the plane.

See the Parameter associations section for possible values that can be specified.

Y2

Specifies the fifth parameter used to define the plane.

Set this parameter to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

Z2

Specifies the sixth parameter used to define the plane.

Set this parameter to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

X3

Specifies the seventh parameter used to define the plane.

Set this parameter to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

Y3

Specifies the eighth parameter used to define the plane.

Set this parameter to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

Z3

Specifies the ninth parameter used to define the plane.

Set this parameter to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

Parameter associations

The table below lists possible values for the **CreationMode**, **X1**, **Y1**, **Z1**, **X2OrD**, **Y2**, **Z2**, **X3**, **Y3**, and **Z3** parameters.

- For specifying the plane

Set unused parameters to **M_DEFAULT**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the plane INQ																	
CreationMode		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	X1																
	Y1																
	Z1																
	X2OrD																
	Y2																
	Z2																
	X3																
	Y3																
	Z3																
M_COEFFICIENTS		Specifies to define the plane using coefficients from the plane equation (Ax + By + Cz + D = 0). Coefficients *A*, *B*, and *C* cannot all be zero. Note that the coefficients will be internally normalized such that A^2 + B^2 + C^2 = 1.															
	X1	Specifies coefficient *A* of the plane equation.	c		e	f	g	a		i		j	k	l	m	n	b
	Y1	Specifies coefficient *B* of the plane equation.	c		e	f	g	a		i		j	k	l	m	n	b
	Z1	Specifies coefficient *C* of the plane equation.	c		e	f	g	a		i		j	k	l	m	n	b

X2OrD	Specifies coefficient *D* of the plane equation.	c		e	f	g	a		i		j	k	l	m	n	b
M_POINT_AND_NORMAL	Specifies to define the plane using a point on the plane and the plane's normal. The plane's normal must have a non-zero magnitude. Note that the plane's normal will be internally converted to a unit vector. Note that if Geometry3dgeoId specifies a previously-defined plane, you can leave the components of its normal vector unchanged, even if that attribute was set using a different creation mode or was modified using the 3D image processing module. To do so, set the corresponding parameter to M_UNCHANGED.	c		e	f	g	a		i		j	k	l	m	n	b
X1	Specifies the X-coordinate of the point on the plane.	c		e	f	g	a		i		j	k	l	m	n	b
Y1	Specifies the Y-coordinate of the point on the plane.	c		e	f	g	a		i		j	k	l	m	n	b
Z1	Specifies the Z-coordinate of the point on the plane.	c		e	f	g	a		i		j	k	l	m	n	b
X2OrD	Specifies the X-component of the normal.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-component of the normal.	c		e	f	g	a		i		j	k	l	m	n	b
Y2	Specifies the Y-component of the normal.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-component of the normal.	c		e	f	g	a		i		j	k	l	m	n	b

Z2	Specifies the Z-component of the normal.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Z-component of the normal.	c		e	f	g	a		i		j	k	l	m	n	b
M_POINT_AND_TWO_VECTORS	Specifies to define the plane using a point on the plane and two vectors parallel to the plane. The two vectors must have non-zero magnitudes and cannot be co-linear.	c		e	f	g	a		i		j	k	l	m	n	b
X1	Specifies the X-coordinate of the point on the plane.	c		e	f	g	a		i		j	k	l	m	n	b
Y1	Specifies the Y-coordinate of the point on the plane.	c		e	f	g	a		i		j	k	l	m	n	b
Z1	Specifies the Z-coordinate of the point on the plane.	c		e	f	g	a		i		j	k	l	m	n	b
X2OrD	Specifies the X-component of the first vector.	c		e	f	g	a		i		j	k	l	m	n	b
Y2	Specifies the Y-component of the first vector.	c		e	f	g	a		i		j	k	l	m	n	b
Z2	Specifies the Z-component of the first vector.	c		e	f	g	a		i		j	k	l	m	n	b
X3	Specifies the X-component of the second vector.	c		e	f	g	a		i		j	k	l	m	n	b
Y3	Specifies the Y-component of the second vector.	c		e	f	g	a		i		j	k	l	m	n	b
Z3	Specifies the Z-component of the second vector.	c		e	f	g	a		i		j	k	l	m	n	b
M_THREE_POINTS	Specifies to define the plane using three points on the plane.	c		e	f	g	a		i		j	k	l	m	n	b

	X1	Specifies the X-coordinate of the first point on the plane.	c		e	f	g	a		i		j	k	l	m	n	b
	Y1	Specifies the Y-coordinate of the first point on the plane.	c		e	f	g	a		i		j	k	l	m	n	b
	Z1	Specifies the Z-coordinate of the first point on the plane.	c		e	f	g	a		i		j	k	l	m	n	b
	X2OrD	Specifies the X-coordinate of the second point on the plane.	c		e	f	g	a		i		j	k	l	m	n	b
	Y2	Specifies the Y-coordinate of the second point on the plane.	c		e	f	g	a		i		j	k	l	m	n	b
	Z2	Specifies the Z-coordinate of the second point on the plane.	c		e	f	g	a		i		j	k	l	m	n	b
	X3	Specifies the X-coordinate of the third point on the plane.	c		e	f	g	a		i		j	k	l	m	n	b
	Y3	Specifies the Y-coordinate of the third point on the plane.	c		e	f	g	a		i		j	k	l	m	n	b
	Z3	Specifies the Z-coordinate of the third point on the plane.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3d11.lib.
DLL	Requires ail11.dll; ail3d11.dll.

M3dgeoPoint

Synopsis

Define an arbitrary 3D point geometry object.

Syntax

```
void M3dgeoPoint(
    AIL_ID Geometry3dgeoId, //out
    AIL_DOUBLE PositionX,   //in
    AIL_DOUBLE PositionY,   //in
    AIL_DOUBLE PositionZ,   //in
    AIL_INT64 ControlFlag   //in
)
```

Description

This function defines an arbitrary 3D point geometry object.

All coordinates are expressed in world units in the working coordinate system.

Parameters

Geometry3dgeoId

Specifies the identifier of the 3D geometry object, previously allocated on the required system using M3dgeoAlloc() with M_GEOMETRY.

PositionX INQ

Specifies the X-coordinate of the point.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the X-coordinate of the point																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-coordinate of the point.	c		e	f	g	a		i		j	k	l	m	n	b

PositionY INQ

Specifies the Y-coordinate of the point.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the Y-coordinate of the point																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b

Value	Specifies the Y-coordinate of the point.	c		e	f	g	a		i		j	k	l	m	n	b

PositionZ INQ

Specifies the Z-coordinate of the point.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the Z-coordinate of the point																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Z-coordinate of the point.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3d11.lib.
DLL	Requires ail11.dll; ail3d11.dll.

M3dgeoRestore

Synopsis

Restore a 3D geometry object or transformation matrix object from disk.

Syntax

```
AIL_ID M3dgeoRestore(
    AIL_CONST_TEXT_PTR FileName,       //in
    AIL_ID SysId,                      //in
    AIL_INT64 ControlFlag,             //in
    AIL_ID *GeometryOrMatrix3dgeoIdPtr //out
)
```

Description

This function restores a 3D geometry object or transformation matrix object that was previously saved to a file, using M3dgeoSave() or M3dgeoStream(). This function restores all of the 3D geometry object 's or transformation matrix object's settings that were in effect when it was saved.

When the restored 3D geometry object or transformation matrix object is no longer required, release it using M3dgeoFree() unless M_UNIQUE_ID was specified during restoration; if M_UNIQUE_ID was specified, the smart identifier manages the 3D geometry object or transformation matrix object's lifetime and you must not manually free it.

Parameters

FileName

Specifies the name and path of the file from which to restore the 3D geometry object or transformation matrix object. The function handles (internally) the opening and closing of the file.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens the **File Open** dialog box from which you can interactively specify the drive, directory, and name of the file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). Typically, registration context files have an M3DGEO extension. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

SysId

Specifies the system on which to restore the 3D geometry object or transformation matrix object.

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

GeometryOrMatrix3dgeoIdPtr

Specifies the address of the variable in which to write the 3D geometry object or transformation matrix object identifier or specifies the data type that the function should use to return the 3D geometry object or transformation matrix object identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated 3D geometry object or transformation matrix object ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated 3D geometry object or transformation matrix object ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_3DGEO_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the 3D geometry object or transformation matrix object (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the 3D geometry object identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored 3D geometry object identifier. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Address in which to write the transformation matrix object identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored transformation matrix object identifier. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b	

Return value

The returned value is the 3D geometry object or transformation matrix object identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_3DGEO_ID*). If restoration fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3d11.lib.
DLL	Requires ail11.dll; ail3d11.dll.

M3dgeoSave

Synopsis

Save a 3D geometry object or transformation matrix object to a file.

Syntax

```
void M3dgeoSave(
    AIL_CONST_TEXT_PTR FileName,    //in
    AIL_ID GeometryOrMatrix3dgeoId, //in
    AIL_INT64 ControlFlag           //in
)
```

Description

This function saves all the information about a previously allocated 3D geometry object or transformation matrix object to disk.

To load a saved context, use either M3dgeoRestore() or M3dgeoStream().

Parameters

FileName

Specifies the name and path of the file in which to save the 3D geometry object or transformation matrix object; it is recommended that you use the M3DGEO file extension for easier use with other Aurora Imaging software products. The function internally handles the opening and closing of this file. If this file already exists, it will be overwritten.

This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens the **File Save As** dialog box from which you can interactively specify the drive, directory, and name of the file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). The recommended extension is M3DGEO. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

GeometryOrMatrix3dgeoId

Specifies the identifier of the 3D geometry object or transformation matrix object to save. These must have been successfully allocated with M3dgeoAlloc() prior to calling this function.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3d11.lib.
DLL	Requires ail11.dll; ail3d11.dll.

M3dgeoSphere

Synopsis

Define a 3D sphere geometry object.

Syntax

```
void M3dgeoSphere(
    AIL_ID Geometry3dgeoId, //out
    AIL_DOUBLE CenterX,     //in
    AIL_DOUBLE CenterY,     //in
    AIL_DOUBLE CenterZ,     //in
    AIL_DOUBLE Radius,      //in
    AIL_INT64 ControlFlag   //in
)
```

Description

This function defines a 3D sphere geometry object, using its center coordinates and radius. You can translate, scale, or transform the resulting sphere, using the 3D image processing module.

You can use the resulting sphere to, for example, crop a point cloud or perform an arithmetic operation on a depth map using the 3D image processing module, or calculate its distance from each point in a point cloud using the 3D metrology module.

If you want to define a sphere geometry object from results obtained in a different module, you can use the copy function of that module.

All coordinates are expressed in world units in the working coordinate system.

> Note that if Geometry3dgeoId specifies a previously-defined sphere, you can leave some of its attributes unchanged, even if that attribute was modified using the 3D image processing module. To do so, set the corresponding parameter to M_UNCHANGED.

Parameters

Geometry3dgeoId

Specifies the identifier of the 3D geometry object, previously allocated on the required system using M3dgeoAlloc() with M_GEOMETRY.

CenterX INQ

Specifies the X-coordinate of the sphere's center.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the X-coordinate of the sphere's center																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b

Value	Specifies the X-coordinate of the sphere's center.	c		e	f	g	a		i		j	k	l	m	n	b

CenterY INQ

Specifies the Y-coordinate of the sphere's center.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the Y-coordinate of the sphere's center																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-coordinate of the sphere's center.	c		e	f	g	a		i		j	k	l	m	n	b

CenterZ INQ

Specifies the Z-coordinate of the sphere's center.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the Z-coordinate of the sphere's center																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Z-coordinate of the sphere's center.	c		e	f	g	a		i		j	k	l	m	n	b

Radius INQ

Specifies the sphere's radius. This parameter cannot be set to a negative value.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the sphere's radius																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_UNCHANGED	Specifies to use the previously-defined value.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the sphere's radius.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3d11.lib.
DLL	Requires ail11.dll; ail3d11.dll.

M3dgeoStream

Synopsis

Load, restore, or save a 3D geometry object or transformation matrix object from/to a file or memory stream.

Syntax

```
void M3dgeoStream(
    AIL_TEXT_PTR MemPtrOrFileName,      //in-out
    AIL_ID SysId,                       //in
    AIL_INT64 Operation,                //in
    AIL_INT64 StreamType,               //in
    AIL_DOUBLE Version,                 //in
    AIL_INT64 ControlFlag,              //in
    AIL_ID *GeometryOrMatrix3dgeoIdPtr, //in-out
    AIL_INT *SizeByteVarPtr             //out
)
```

Description

This function can load, restore, or save a 3D geometry object or transformation matrix object from/to a file or memory stream.

To inquire the number of bytes necessary to save a 3D geometry object or transformation matrix object to a memory stream, you should first call this function (**M3dgeoStream**()) with M_INQUIRE_SIZE_BYTE.

The content saved to memory stream is equivalent to the content saved to file. This function is equivalent to a file saved using M3dgeoSave().

You can use this and other Aurora Imaging Library stream functions, for example, to save all required Aurora Imaging Library objects, as well as any other custom data, for your application to a memory stream. Once in a memory stream, you can write the stream to a single file or transfer it over a network. You are responsible for concatenating the streams and for saving the stream to file.

Using **M3dgeoStream**(), you can choose to save a backwards-compatible version of the 3D geometry object or transformation matrix object, which will work using a version of Aurora Imaging Library that is up to one major release older than the current version (depending on which version is specified). However, all settings and features unique to the higher version will be ignored when restored using the lower version. Besides saving backwards-compatible versions, you can also load or restore 3D geometry objects or transformation matrix objects saved using any Aurora Imaging Library version with which this function was available. Settings that do not exist in the lower version will be filled with default values when the 3D geometry object or transformation matrix object is loaded or restored.

Parameters

MemPtrOrFileName
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_TEXT_PTR [optionally, in C++: a reference to an AIL_STRING]
- Optionally, in C++: a reference to a std::vector of type AIL_UINT8

Specifies the file or memory stream.

See the Parameter associations section for possible values that can be returned/specified.

SysId

Specifies the system on which to restore the 3D geometry object or transformation matrix object.

See the Parameter associations section for possible values that can be specified.

Operation

Specifies the operation to perform.

See the Parameter associations section for possible values that can be specified.

StreamType

Specifies the type of stream in which to store/from which to restore the 3D geometry object or transformation matrix object. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of stream																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FILE	Specifies a file stream.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEMORY	Specifies a memory stream. You are responsible for allocating a block of memory for the stream.	c		e	f	g	a		i		j	k	l	m	n	b

Version

Specifies the Aurora Imaging Library version of the 3D geometry object or transformation matrix object.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

GeometryOrMatrix3dgeoIdPtr

Specifies the address of the variable in which to write or from which to read the identifier of the 3D geometry object or transformation matrix object. If the M_RESTORE operation is not successful, **M_NULL** is returned.

See the Parameter associations section for possible values that can be returned/specified.

SizeByteVarPtr

Specifies the address of the variable in which to write the size of the 3D geometry object or transformation matrix object, in bytes.

If the size is not required, you can set this parameter to **M_NULL**.

> Note that the size of the 3D geometry object or transformation matrix object will vary depending on the Aurora Imaging Library version specified.

Parameter associations

The table below lists possible values for the **MemPtrOrFileName**, **SysId**, **Operation**, **Version**, and **GeometryOrMatrix3dgeoIdPtr** parameters.

- For performing the stream operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For performing the stream operation.																	
Operation	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
MemPtrOrFileName - Possible values passed or returned																
SysId																
Version																
GeometryOrMatrix3dgeoIdPtr - Possible values passed or returned																
M_INQUIRE_SIZE_BYTE	Inquires the number of bytes required to save a 3D geometry object or transformation matrix object to memory stream. This operation is not supported when the StreamType parameter is set to M_FILE.															
MemPtrOrFileName	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version	Specifies the Aurora Imaging Library version with which the 3D geometry object or transformation matrix object must be compatible when saved. The size of a 3D geometry object or transformation matrix object might differ from one Aurora Imaging Library version to another.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b

		M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
	GeometryOrMatrix3dgeoIdPtr		Specifies the address of the variable containing the identifier of the 3D geometry object or transformation matrix object.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOAD			Loads the content of a specified file or memory stream into a previously allocated 3D geometry object or transformation matrix object.	c		e	f	g	a		i		j	k	l	m	n	b
	MemPtrOrFileName		Specifies the file or memory stream from which to load the 3D geometry object or transformation matrix object.	c		e	f	g	a		i		j	k	l	m	n	b
		M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b

		AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. 3D geometry objects or transformation matrix objects typically have an M3DGEO file extension. The function handles (internally) the opening and closing of the file. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
		MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
	SysId		This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
	Version		This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
	GeometryOrMatrix3dgeoIdPtr		Specifies the address of the variable containing the identifier of the 3D geometry object or transformation matrix object. All previous settings are overwritten with those of the loaded 3D geometry object or transformation matrix object, or are set to the default value if the settings didn't exist in the version of Aurora Imaging Library from which the 3D geometry object or transformation matrix object is being loaded.	c		e	f	g	a		i		j	k	l	m	n	b

M_RESTORE	Restores a 3D geometry object or transformation matrix object from a file or memory stream and assigns it an Aurora Imaging Library identifier.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName	Specifies the file or memory stream from which to restore the 3D geometry object or transformation matrix object.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. 3D geometry objects or transformation matrix objects typically have an M3DGEO file extension. The function handles (internally) the opening and closing of the file. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	Specifies the system on which to allocate the 3D geometry object or transformation matrix object.	c		e	f	g	a		i		j	k	l	m	n	b

		M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
		System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
	Version		This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
	GeometryOrMatrix3dgeoIdPtr		Specifies the address of the variable in which to write the identifier of the 3D geometry object or transformation matrix object. If the operation is not successful, **M_NULL** is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE			Saves a 3D geometry object or transformation matrix object to a specified file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
	MemPtrOrFileName		Specifies the file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
		M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
		AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. For easier use with other Aurora Imaging software products, when saving a 3D geometry object or transformation matrix object to a file, use the M3DGEO file extension. The function handles (internally) the opening and closing of the file. If the file already exists, it will be overwritten. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

		MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which to write, and the block of memory must be large enough to stream the entire object. To determine the required size, call this function with M_INQUIRE_SIZE_BYTE. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
	SysId		This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
	Version		Specifies the Aurora Imaging Library version with which the 3D geometry object or transformation matrix object must be compatible when saved.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b

	GeometryOrMatrix3dgeoIdPtr	Specifies the address of the variable containing the identifier of the 3D geometry object or transformation matrix object to save.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3d11.lib.
DLL	Requires ail11.dll; ail3d11.dll.